Embedded Operating System and Architecture

Stridi Simone

Anno Accademico 2026/2027

Indice

1 Progetto **1**

1.1 In breve . . . 1
1.2 Squadre, versione e iscrizione . . . 1
1.2.1 Il progetto nell'esame . . . 1
1.2.2 Formazione delle squadre . . . 2
1.2.3 Versione assegnata . . . 2
1.3 Da dove si parte . . . 3
1.3.1 Il sistema in due tempi . . . 3
1.3.2 Che cosa è dato e che cosa scrive la squadra . . . 4
1.4 Le idee che servono . . . 5
1.4.1 Perché un programma caricato va corretto . . . 5
1.4.2 Le parti di un programma . . . 5
1.4.3 Magic number e versione . . . 6
1.4.4 Le parti di un file ELF che servono . . . 6
1.4.5 Indirizzi Thumb . . . 6
1.4.6 Protezione della memoria e W^X . . . 6
1.4.7 Glossario . . . 7
1.5 Che cosa bisogna fare: i requisiti . . . 7
1.5.1 R1: programmi compilati separatamente dal kernel . . . 7
1.5.2 R2: un host tool impacchetta i programmi . . . 8
1.5.3 R3: il kernel legge l'immagine sul posto . . . 8
1.5.4 R4: una shell per elencare e avviare i programmi . . . 9
1.5.5 R5: i programmi si avviano con system call . . . 9
1.5.6 R6: il loader mette il programma in uno slot libero e lo fa funzionare lì . . . 9
1.5.7 R7: un programma può toccare solo il proprio slot . . . 10
1.5.8 R8: la fine di un programma libera il suo slot . . . 10
1.5.9 R9: un input sbagliato non rompe mai il sistema . . . 11
1.5.10 B1 (solo big): il kernel carica file ELF . . . 11
1.5.11 B2 (solo big): codice e dati protetti separatamente . . . 11
1.5.12 Tabella riassuntiva . . . 12
1.6 Formati e messaggi esatti . . . 12
1.6.1 L'immagine IMG1 . . . 12
1.6.2 Il file programma SFT1 (solo small) . . . 14
1.6.3 I file ELF dei programmi e l'host tool . . . 16
1.6.4 Mappa di memoria e slot . . . 17
1.6.5 System call . . . 19
1.6.6 Input e output della shell . . . 22

1.6.7 Il codice dato da usare . . . 23
1.7 Come funziona il sistema finito . . . 25
1.7.1 Build sul PC . . . 25
1.7.2 Avvio . . . 25
1.7.3 Shell e comandi . . . 25
1.7.4 Caricamento (SVC 13) . . . 25
1.7.5 Esecuzione . . . 26
1.7.6 Fine del programma . . . 27
1.8 Vincoli e divieti . . . 28
1.8.1 Che cosa non si può fare . . . 28
1.8.2 Che cosa si può cambiare . . . 29
1.8.3 Che cosa è fisso e che cosa decide la squadra . . . 30
1.8.4 Regole di stile e di codice . . . 30
1.9 Comandi . . . 31
1.9.1 Build ed esecuzione . . . 31
1.9.2 Host tool . . . 31
1.9.3 Test . . . 31
1.9.4 Ispezione e debug . . . 32
1.10 Piano di lavoro consigliato . . . 32
1.11 Test e verifica . . . 33
1.11.1 Come controlla il professore . . . 33
1.11.2 Il test kit . . . 34
1.11.3 I test . . . 34
1.11.4 Come verificare da soli prima di consegnare . . . 35
1.12 Consegna . . . 36
1.13 Valutazione . . . 37
1.14 Errori comuni e consigli del professore . . . 39
1.15 Checklist finale prima della consegna . . . 40
1.16 Mappa dei file di project-base — scratch-OS (Cortex-M3, QEMU mps2-an385) . . . 42
1.16.1 Build e linker . . . 42
1.16.2 Boot e definizioni hardware . . . 43
1.16.3 Applicazione di base (`src`) . . . 44
1.16.4 Driver . . . 45
1.16.5 Kernel . . . 46
1.16.6 Libreria lato utente (`lib`) . . . 51
1.16.7 Programmi . . . 52
1.16.8 Mappa finale delle dipendenze . . . 53

Capitolo 1

Progetto

Questo capitolo riassume il progetto del corso a partire da due file del professore:

- `Projects.md`: regole generali, squadre e valutazione;
- `assignment.md`: la consegna completa, collegata da `Projects.md` come `project/assignment.md`.

Le frasi marcate *Da chiarire con il professore* indicano punti che i due file non definiscono in modo univoco.

1.1 In breve

Oggi in scratch-OS ogni task è linkato con il kernel, e per aggiungerne uno si ricompila tutto. Con il progetto i programmi diventano **file separati**, raccolti in un'**immagine** read-only in FLASH, e il kernel ne carica uno solo quando l'utente lo chiede, come in un sistema operativo general purpose. Si scrivono tre pezzi: una **shell** sulla UART (`help`, `ls`, `run <name>`); un **loader** nel kernel, che copia il programma in uno **slot** libero di RAM, ne corregge gli indirizzi (relocation), lo protegge con la MPU e lo avvia come task; un **host tool** in Python che impacchetta i programmi, compilati a parte. A fine programma, o dopo un fault, il codice dato libera lo slot e la shell torna al prompt. Un unico test kit controlla l'output di tutte le squadre, byte per byte.

Questa è la sessione che il sistema finito deve produrre:

```
scratch> ls
print_plain
thumb_ptr
scratch> run print_plain
loader: print_plain slot=0 base=0x20002000 id=4
print_plain: hello from a loaded program
program print_plain exited
scratch>
```

1.2 Squadre, versione e iscrizione

1.2.1 Il progetto nell'esame

Il progetto è **opzionale** e si fa a squadre. Per chi lo fa, sostituisce la Parte 1.2 dell'esame finale, descritta in `Exam.md`. *Da chiarire con il professore:* `Exam.md` non è

tra i file allegati, quindi qui mancano il contenuto della Parte 1.2 e il modo in cui i punti del progetto si combinano con il resto dell'esame.

Il progetto vale da 0 a 14 punti (dettagli in). Si discute con una presentazione orale di 20 minuti, a cui partecipano tutti i membri, seguita da domande sui contributi individuali e da una demo dal vivo, dove applicabile.

Ci si può iscrivere al progetto **una sola volta**, nel primo anno accademico in cui ci si iscrive al corso. Conta l'anno registrato sul portale della didattica, non l'anno in cui si seguono fisicamente le lezioni.

Tutte le squadre fanno lo stesso progetto: il loader di programmi a run time e la shell di scratch-OS.

1.2.2 Formazione delle squadre

Una squadra ha da 3 a 6 membri. La scadenza per formarla è il 16 ottobre.

I membri si inseriscono nel Google Spreadsheet indicato in `Projects.md`:

`https://docs.google.com/spreadsheets/d/15x3NH4fFHiAvLpW22a94eos01w1pQFR_wGFd4iGAVNY/edit?usp=sharing`

Il foglio elenca tutti gli iscritti e controlla in automatico l'anno di iscrizione. Se il controllo fallisce, non si manda una email al professore: si aggiunge nel foglio una breve nota che spiega la situazione (per esempio: iscritto l'anno precedente, ma solo al secondo semestre, quindi senza la possibilità di seguire il corso). Il professore esamina tutte le eccezioni dopo aver raccolto il quadro completo. Il foglio è il punto unico per tutte le informazioni su squadre ed eccezioni.

1.2.3 Versione assegnata

La versione dipende dal numero di membri quando viene assegnato il progetto, e poi non cambia più, anche se la squadra cambia dimensione.

Membri	Versione
3 o 4	Small
5 o 6	Big: tutto il comportamento comune più i due extra B1 e B2

- **Versione small**: l'host tool converte ogni programma ELF nel formato semplice SFT1; il kernel carica solo file SFT1 e non legge mai ELF; una sola regione MPU dà al programma lettura, scrittura ed esecuzione su tutto il suo slot.

- **Versione big**: tutto ciò che chiede la small tranne la conversione in SFT1, più due extra: il kernel legge direttamente i file ELF, che l'immagine contiene invariati (B1), e la MPU applica la regola W^X (B2).

Le due versioni valgono al massimo 14 punti. Alla big si tolgono punti se manca un extra ().

1.3 Da dove si parte

Tutte le squadre partono da `scratch-os/project-base/` e girano su QEMU, board `mps2-an385` (ARM Cortex-M3). I percorsi citati si riferiscono al repository pubblicato di scratch-OS, salvo indicazione diversa. La base compila e parte con `make run`, ma non ha né shell né loader. Il suo file immagine è un segnaposto volutamente non valido come IMG1 ().

Il .md non confronta `project-base` con la base della week-05. *Da chiarire con il professore:* che cosa cambia rispetto alla base della week-05.

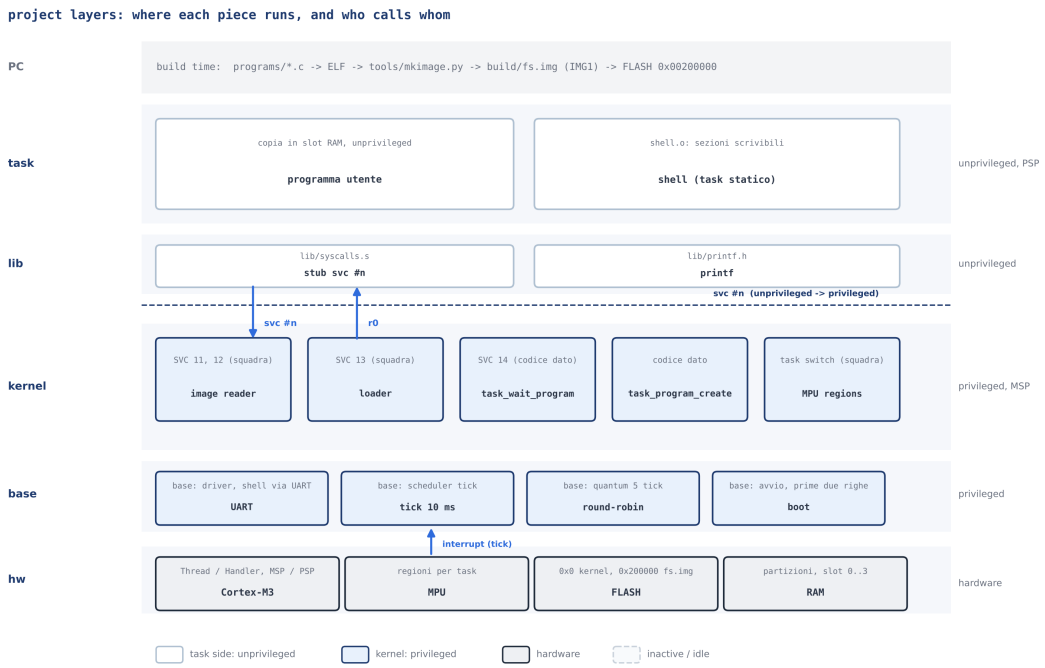


Figura 1.1: Stratificazione del sistema: i programmi utente girano unprivileged nel proprio slot di RAM, la shell è un task statico e il kernel privilegiato contiene loader, SVC 11–14 e MPU. Le frecce `svc` portano il controllo verso il kernel, il ritorno verso i programmi, e il tick arriva dall'hardware.

1.3.1 Il sistema in due tempi

Il lavoro si divide tra il PC (build time) e la board emulata da QEMU (run time):

1. sul PC, ogni `programs/*.c` passa dal compilatore e dal linker dei programmi e diventa un file programma;
2. sempre sul PC, l'host tool (`mkimage.py`) impacchetta tutti i file programma in `fs.img`, in formato IMG1;
3. in QEMU, la FLASH da `0x00000000` contiene kernel e shell, mentre la FLASH da `0x00200000` contiene `fs.img`, in sola lettura;
4. a run time la catena è: UART → shell → SVC → ricerca del programma → slot di RAM libero → copia, BSS a zero, relocation → MPU e creazione del task;

5. il programma gira unprivileged; quando finisce (return, exit o fault) lo slot si libera e la shell si risveglia.

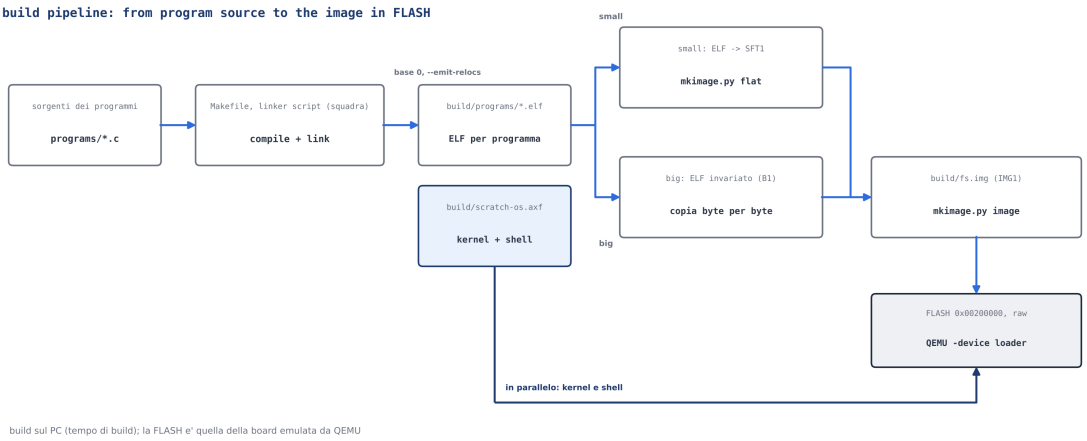


Figura 1.2: Dal sorgente all'immagine in FLASH: ogni programma è compilato e linkato a indirizzo 0 con `-emit-relocs`, e l'host tool crea `build/fs.img` in formato IMG1. Nella small i programmi entrano come SFT1, nella big come ELF invariati.

1.3.2 Che cosa è dato e che cosa scrive la squadra

Il codice dato, da usare e non da riscrivere, comprende:

- i task record per quattro programmi caricati e i task id;
- l'attesa della fine di un programma e il report di come è finito;
- la gestione dell'exit e dei fault dei programmi;
- la system call 14.

La sua interfaccia è descritta in . La squadra scrive invece:

- la shell e le system call da 11 a 13;
- la lettura dell'immagine e il controllo dei file programma;
- la copia e la relocation dei programmi;
- le impostazioni MPU di uno slot;
- startup texttt, linker script e regole di build dei programmi;
- l'host tool `tools/mkimage.py`.

1.4 Le idee che servono

1.4.1 Perché un programma caricato va corretto

Quando il linker costruisce un programma, fissa l'indirizzo di ogni funzione e di ogni variabile globale. Per un programma caricato a run time questo indirizzo non si conosce in anticipo: dipende da quale area di RAM è libera in quel momento. Per questo il programma si linka come se partisse dall'indirizzo `0`, e il loader lo copia dall'immagine in uno slot di RAM prima di eseguirlo.

La maggior parte delle istruzioni non dipende dall'indirizzo: un salto del tipo "avanti di 40 byte" funziona ovunque. Alcuni valori però contengono un indirizzo completo, per esempio un puntatore a una variabile globale o una tabella di puntatori a funzione. Questi valori sono giusti solo per l'indirizzo `0`: se il loader copia il programma a `0x20003000`, deve sommare `0x20003000` a ciascuno di essi.

Questa correzione si chiama **relocation** e l'elenco dei punti da correggere è la **relocation list**. Il linker produce questo elenco quando si linka con l'opzione `-emit-relocs`. Il Cortex-M3 non ha MMU (niente memoria virtuale), quindi non c'è un modo automatico per far girare un programma a un altro indirizzo: bisogna correggerlo.

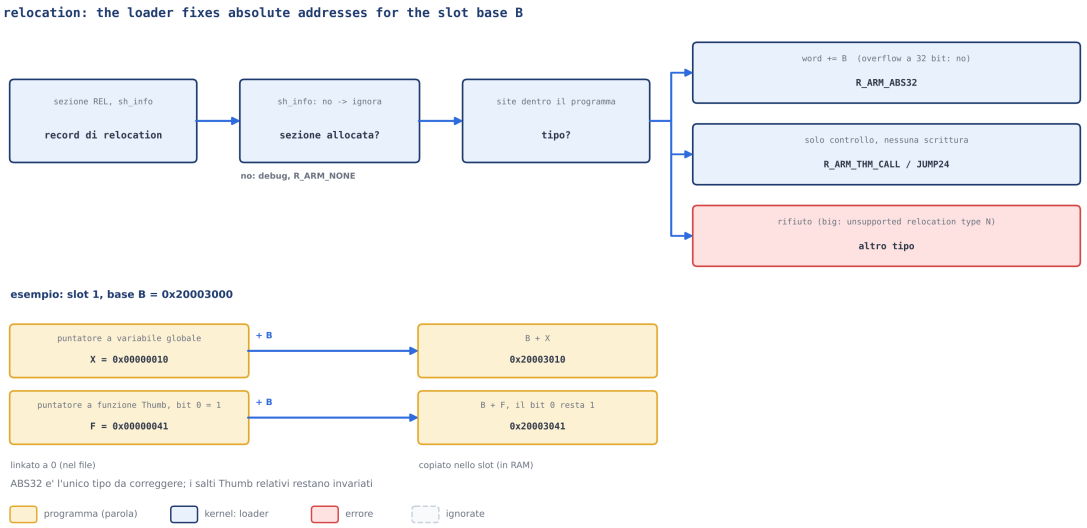


Figura 1.3: Correzione degli indirizzi: ogni record di una sezione allocata viene applicato secondo il suo tipo. Nell'esempio il loader sposta i puntatori di un programma nello slot 1, sommando la base `0x20003000`.

1.4.2 Le parti di un programma

Un programma in memoria ha quattro parti:

- **texttt**: istruzioni e costanti read-only, per esempio le stringhe;
- **data**: variabili globali con un valore iniziale, come `int x = 5;`;
- **BSS**: variabili globali che partono da zero, come `int y;`. Il file non contiene questi byte ma solo la loro dimensione, e il loader riempie l'area di zeri;
- **stack**: usato mentre il programma gira.

1.4.3 Magic number e versione

Un file è solo una sequenza di byte. Per riconoscerne il formato, quasi tutti i formati binari iniziano con pochi byte fissi, il **magic number** (o *magic*). Chi legge il file confronta prima questi byte: se non coincidono con quelli attesi, il file non è in quel formato (o è danneggiato) e viene rifiutato subito.

Ogni file ELF, per esempio, inizia con i quattro byte `7f 45 4c 46` (`0x7f` seguito dalle lettere `ELF`). Il progetto usa due formati propri: un'immagine inizia con le quattro lettere ASCII `IMG1`, un file programma `SFT1` con le lettere `SFT1`. Subito dopo la magic, entrambi i formati contengono un numero di **versione**, così il formato può cambiare in futuro senza essere confuso con il vecchio. Nel progetto la versione vale sempre `1`.

1.4.4 Le parti di un file ELF che servono

Il linker di GCC produce file ELF. Non serve conoscere tutto lo standard, solo queste parti:

- l'**ELF header**, all'inizio del file: contiene la magic, il tipo di file, l'entry point e la posizione delle due tabelle seguenti;
- la **program header table**, che descrive i **segmenti**, cioè i pezzi del file da mettere in memoria per eseguire il programma. Un segmento di tipo `PT_LOAD` va caricato;
- la **section header table**, che descrive le **sezioni**, i pezzi più fini su cui lavora il linker: `.text` (codice), `.rodata` (costanti), `.data` e `.bss`. Una sezione è **allocated** quando occupa memoria mentre il programma gira (le informazioni di debug, per esempio, non lo sono). Una sezione di tipo `SHT_NOBITS` occupa memoria ma nessun byte nel file: è così che è memorizzato il BSS;
- le sezioni di relocation di tipo `REL`: ognuna contiene i record di relocation di un'altra sezione, indicata dal suo campo `sh_info`. Ogni record dà una posizione (il *site*), un simbolo e un **tipo di relocation**, che dice come calcolare l'indirizzo.

Il progetto usa solo tre tipi di relocation: `R_ARM_ABS32`, un indirizzo completo a 32 bit, e `R_ARM_THM_CALL` e `R_ARM_THM_JUMP24`, salti Thumb relativi alla posizione corrente. Di solito il linker toglie questi record dal file finale; `-emit-relocs` gli chiede di tenerli, così il loader li può usare. Tutte queste parti si vedono con `arm-none-eabi-readelf` ().

1.4.5 Indirizzi Thumb

Il Cortex-M3 esegue solo istruzioni Thumb. In ARM l'indirizzo di una funzione Thumb ha il bit più basso (bit 0) a 1. Quando il loader corregge un puntatore a funzione sommando l'indirizzo base, il bit 0 deve restare com'è.

1.4.6 Protezione della memoria e W^X

I programmi caricati girano unprivileged. La MPU deve impedire a un programma di toccare il kernel o la memoria di un altro programma. La versione big chiede anche

la regola **W^X** ("write xor execute"): un'area di memoria può essere scrivibile o eseguibile, mai le due cose insieme. Con W^X un programma non può sovrascrivere il proprio codice né eseguire byte che stanno nei suoi dati o nello stack.

1.4.7 Glossario

Termine	Significato nel progetto
Slot	Area fissa di RAM da 4 KiB in cui vive un programma caricato. Gli slot sono quattro.
Magic number	Byte fissi all'inizio di un file che ne identificano il formato.
Segmento, sezione	Pezzi di un file ELF: i segmenti dicono che cosa caricare, le sezioni sono i pezzi più fini del linker (codice, costanti, dati, BSS, debug).
Sezione allocated	Sezione che occupa memoria mentre il programma gira.
Immagine, IMG1	File read-only in FLASH che contiene tutti i file programma e una directory con i loro nomi.
SFT1	Formato semplice dei file programma della versione small, prodotto dall'host tool della squadra.
ELF	Formato standard dei file prodotti dal linker di GCC.
Relocation	Parola del programma che contiene un indirizzo e che il loader deve correggere.
BSS	Variabili globali che partono da zero e non sono memorizzate nel file.
W^X	Un'area di memoria è scrivibile oppure eseguibile, non entrambe le cose.
Task id	Numero che identifica un programma caricato in esecuzione. Combina il numero dello slot con un contatore, la **generation**, che cresce a ogni riuso dello slot: così un id vecchio non coincide mai con un nuovo programma nello stesso slot.

1.5 Che cosa bisogna fare: i requisiti

I requisiti dicono *che cosa* deve fare il sistema, non in che ordine lavorare: il .md precisa che non sono un piano di lavoro e che ordine e divisione dei compiti li decide la squadra (una proposta di ordine è in). R1–R9 valgono per entrambe le versioni, B1 e B2 solo per la big. Per la propria versione sono tutti obbligatori; opzionali sono solo il progetto stesso e gli extra in fondo alla tabella riassuntiva ().

1.5.1 R1: programmi compilati separatamente dal kernel

Che cosa chiede Startup texttt, linker script e regole di build che compilano ogni programma di `programs/` in un proprio file ELF, linkato all'indirizzo `0` con `-emit-relocs`. Un programma non condivide codice con il kernel: ognuno include il proprio startup texttt, i propri stub delle system call, `printf` e ogni helper del compilatore che gli serve. `_start` chiama `main`, e tornare da `main` chiude il programma come se avesse chiamato `sys_exit()`. La build crea i programmi e `build/fs.img` dai sorgenti, con GCC ARM bare-metal, ogni volta nello stesso modo.

Perché serve Un programma caricato a run time non conosce il proprio indirizzo: linkato a `0` e con le relocation conservate, può essere spostato in qualunque slot. Non condividendo codice con il kernel, è un file davvero autonomo.

Dove si interviene I sorgenti in `programs/`; startup texttt e linker script dei programmi (nomi scelti dalla squadra); il Makefile (dettagli scelti dalla squadra); gli stub di `lib/syscalls.s` e `lib/printf.h`. Nell'esempio di debug del .md un programma compilato si trova in `build/programs/thumb_ptr.elf`. Dettagli in e .

Come si verifica Con `arm-none-eabi-readelf` si controlla che ogni ELF sia di tipo `ET_EXEC`, con entry point Thumb, almeno un segmento `PT_LOAD` e le sezioni `REL`. I test `normal_return` ed `explicit_exit` controllano il ritorno da `main` e la chiamata a `sys_exit`. Una build da zero deve ricreare programmi e immagine.

1.5.2 R2: un host tool impacchetta i programmi

Che cosa chiede Scrivere `tools/mkimage.py`. Lo strumento costruisce `build/fs.img`, in formato IMG1, da una lista di programmi, e rifiuta l'input sbagliato con un messaggio d'errore chiaro. Le squadre small implementano anche la conversione da ELF a SFT1.

Perché serve Il kernel deve trovare tutti i programmi in un solo file in FLASH, in un formato fisso che conosce anche il test kit.

Dove si interviene `tools/mkimage.py`; la ricetta `make image` della base, da adattare (); il risultato `build/fs.img`. Dettagli in , e .

Come si verifica In caso di successo lo strumento stampa esattamente i messaggi previsti (); con un input sbagliato scrive `mkimage.py: error: <reason>` su standard error ed esce con stato 2, senza scrivere output. In QEMU, `ls` elenca i programmi impacchettati.

1.5.3 R3: il kernel legge l'immagine sul posto

Che cosa chiede Il kernel trova l'immagine all'indirizzo FLASH `0x00200000`, la controlla e cerca i programmi per nome. La legge dov'è: non ci scrive mai e non la copia mai intera in RAM.

Perché serve L'immagine emula un disco esterno in sola lettura; copiarla sprecherebbe RAM, e l'efficienza è valutata (niente copie inutili dell'immagine).

Dove si interviene Il codice del kernel che legge e controlla l'immagine (modulo e nomi scelti dalla squadra), che la trova tramite il simbolo del linker `__fs_image_start`. Dettagli in .

Come si verifica `ls` elenca i nomi nell'ordine della directory. Con un'immagine non valida, come il segnaposto della base, `ls` stampa `error: no valid image`, `run` stampa `error: program cannot be loaded` e la shell continua a funzionare.

1.5.4 R4: una shell per elencare e avviare i programmi

Che cosa chiede La shell è un task statico che legge righe dalla UART e accetta i comandi `help`, `ls` e `run <name>`. Stampa esattamente il prompt e i messaggi di . Mentre un programma gira, aspetta senza stampare il prompt.

Perché serve È l'interfaccia verso l'utente e verso il test kit, che confronta ogni carattere stampato.

Dove si interviene Il sorgente della shell (le sue sezioni scrivibili vengono da `shell.o`); la configurazione del task con `task_project_configure()` e `task_configure_shell()`; i simboli del linker della shell ().

Come si verifica Le trascrizioni del test kit contengono prompt, eco e messaggi di ogni test. A mano: una riga vuota, `run` da solo o `help` seguito da altri caratteri devono stampare `error: unknown command`; una riga lunga si ferma a 31 caratteri.

1.5.5 R5: i programmi si avviano con system call

Che cosa chiede La shell usa le system call da 11 a 14 per contare i programmi, leggerne i nomi, avviarne uno e aspettarne la fine. I risultati devono essere esattamente quelli di .

Perché serve La shell non tocca direttamente immagine e task: passa dal kernel con una SVC. Le stesse chiamate le usa anche un programma caricato che avvia un altro programma (test `nonzero_slot_relocation`).

Dove si interviene Le SVC 11, 12 e 13 nel kernel (la 14 è già data); gli stub lato utente, come quelli di `lib/syscalls.s`. *Da chiarire con il professore:* il .md non dice se `lib/syscalls.s` contiene già gli stub delle SVC 11–14 o se vanno aggiunti.

Come si verifica Ogni caso d'errore restituisce il codice giusto (`-1`, `-2`, `-3`, `-4`) e la shell stampa il messaggio corrispondente. Nel test `nonzero_slot_relocation` il programma padre riceve 0.

1.5.6 R6: il loader mette il programma in uno slot libero e lo fa funzionare lì

Che cosa chiede Per `run <name>` il kernel sceglie il primo slot libero, copia codice e dati nello slot, riempie di zeri il BSS, somma l'indirizzo dello slot a ogni parola della relocation list, controlla entry point e stack, e solo allora crea il task.

Perché serve È il cuore del progetto ed è il criterio che vale di più (2.50 punti).

Dove si interviene Il loader nel kernel (scritto dalla squadra); le funzioni date `task_program_slot_free(slot)` e `task_program_create(slot, name, entry, stack_top, executable)`; i simboli del linker degli slot. Dettagli in (small), (big) e .

Come si verifica La riga `loader:` riporta slot, base e id corretti. I test `global_state` (globali, BSS e costanti), `thumb_ptr` (puntatore a funzione Thumb), `nonzero_slot_relocation` (relocation nello slot 1) e `too_large` (programma troppo grande) controllano i casi principali.

Da chiarire con il professore: il .md non elenca uno per uno i controlli sullo stack. Da si sa che il programma occupa al massimo i 3072 byte bassi dello slot, che lo stack occupa i 1024 byte alti, che lo stack pointer iniziale è la fine dello slot e che i frame iniziali usano i 64 byte in cima.

1.5.7 R7: un programma può toccare solo il proprio slot

Che cosa chiede Un programma caricato esegue sempre dal proprio slot di RAM, dove il loader l'ha copiato: il file nell'immagine è solo la sorgente della copia. I programmi girano unprivileged e la MPU permette a un programma il proprio slot e nient'altro in RAM: né la RAM del kernel né altri slot. La FLASH resta leggibile ed eseguibile per tutti i task unprivileged, come nella base, perché la shell è un task statico con il codice in FLASH; la FLASH non si può scrivere, quindi la protezione non si indebolisce.

Perché serve Un programma con un bug non deve poter corrompere il kernel o gli altri programmi: il suo errore diventa un fault recuperabile.

Dove si interviene Le impostazioni MPU di uno slot; l'argomento `executable` di `task_program_create`; nella big anche il setup MPU fatto a ogni task switch. Dettagli in .

Come si verifica Test `kernel_store`: il programma scrive a `0x20000000` e il sistema stampa `MemManage in loaded program: CFSR=0x82 address=0x20000000`; poi `print_plain` riusa lo slot 0 con id 8. Il professore consiglia anche di controllare i permessi risultanti dopo ogni task switch.

1.5.8 R8: la fine di un programma libera il suo slot

Che cosa chiede Quando un programma torna da `main`, chiama `sys_exit()` o provoca un fault recuperabile, il sistema stampa il messaggio corrispondente, libera subito lo slot e sveglia la shell, che mostra di nuovo il prompt. Questo comportamento si ottiene usando bene il codice dato (), non scrivendone una versione propria.

Perché serve Gli slot sono solo quattro e devono tornare subito disponibili; il codice comune garantisce lo stesso comportamento in tutte le squadre.

Dove si interviene Lato squadra: lo startup texttt (ritorno da `main`) e la shell, che chiama la SVC 14. Lato codice dato, da non toccare: `task_wait_program`, il codice della SVC 14, `task_finish_current`, `task_fault_current`, `task_current_is_program`, l'entry dei fault e il dispatcher.

Come si verifica Test `normal_return`, `explicit_exit` e `kernel_store` (dopo il fault lo slot 0 viene riusato con id 8).

1.5.9 R9: un input sbagliato non rompe mai il sistema

Che cosa chiede Un'immagine danneggiata, un file programma danneggiato o un argomento sbagliato di una system call vengono rifiutati con l'errore giusto, prima che un solo byte del programma sia copiato o eseguito. Il kernel non va in crash, nessuno slot cambia e la shell continua a funzionare.

Perché serve Il kernel non può fidarsi di dati che vengono da fuori: l'immagine e gli argomenti passati da codice unprivileged.

Dove si interviene I controlli sull'immagine (), sui file SFT1 () o ELF () e sugli argomenti delle system call ().

Come si verifica Test `bad_magic`, `bad_header`, `too_large` e `bad_reloc`: dopo il rifiuto, `help` mostra che la shell funziona ancora. Nella big anche `unsupported_reloc` e `truncated_elf`.

1.5.10 B1 (solo big): il kernel carica file ELF

Che cosa chiede L'immagine contiene ogni file ELF esattamente come l'ha prodotto il linker, e l'host tool si limita a impacchettare. È il kernel a controllare l'ELF, a trovare le parti da caricare e le relocation, e ad applicarle.

Perché serve È il lavoro di un loader vero: niente formato intermedio preparato sul PC.

Dove si interviene Il lettore ELF nel kernel (struttura scelta dalla squadra) e il comando `image` di `tools/mkimage.py`, che copia gli ELF byte per byte. Dettagli in .

Come si verifica La suite big esegue i dieci test comuni con file ELF, più `unsupported_reloc` (stampa `loader: unsupported relocation type 3`) e `truncated_elf`. Se B1 manca o non funziona: penalità di -2.00 punti.

1.5.11 B2 (solo big): codice e dati protetti separatamente

Che cosa chiede La regola W^X: il codice si può leggere ed eseguire ma non scrivere; dati e stack si possono leggere e scrivere ma non eseguire. Un programma che scrive nel proprio codice o salta nei propri dati si ferma con un memory fault recuperabile, come in R8.

Perché serve Impedisce a un programma di modificare il proprio codice e di eseguire byte scritti nei dati o nello stack.

Dove si interviene Le regioni MPU (quali usare lo decide la squadra). Le squadre big possono aggiungere campi ai task record ed estendere il setup MPU fatto a ogni task switch. L'argomento `executable` da solo non basta per W^X.

Come si verifica Test `wx_write_texttt` (`MemManage in loaded program: CFSR=0x82 address=0x20002280`) e `wx_exec_data` (`MemManage in loaded program: CFSR=0x1`); in entrambi un programma corretto riusa poi lo slot 0 con id 8. Se B2 manca, anche in una sola direzione: penalità di -1.00 punti, contata una volta sola.

1.5.12 Tabella riassuntiva

Requisito	Obbligatorio?	File coinvolti	Come si verifica
R1 Build dei programmi	Sì, small e big	`programs/`, startup texttt, linker script, Makefile, `lib/syscalls.s`	`arm-none-eabi-readelf`; `normal_return`, `explicit_exit`
R2 Host tool	Sì; `flat` solo small	`tools/mkimage.py`, `build/fs.img`	Messaggi esatti; errori con stato 2
R3 Immagine sul posto	Sì	Lettore dell'immagine nel kernel	`ls`; `error: no valid image`
R4 Shell	Sì	Sorgente di `shell.o`, simboli del linker della shell	Trascrizioni del test kit
R5 System call 11–14	Sì	SVC 11–13 nel kernel, stub	Codici di ritorno; `nonzero_slot_relocation`
R6 Loader	Sì	Loader, `task_program_create`	`global_state`, `thumb_ptr`, `too_large`
R7 Isolamento MPU	Sì	Codice MPU	`kernel_store`
R8 Fine e riuso dello slot	Sì, con il codice dato	Startup texttt, shell; codice dato di `kernel/task.h`	`normal_return`, `explicit_exit`, `kernel_store`
R9 Rifiuto dell'input sbagliato	Sì	Tutti i controlli del kernel	`bad_magic`, `bad_header`, `too_large`, `bad_reloc`
B1 ELF nel kernel	Solo big (altrimenti −2.00)	Lettore ELF, comando `image`	`unsupported_reloc`, `truncated_elf`
B2 W^X	Solo big (altrimenti −1.00)	Codice MPU, task record	`wx_write_texttt`, `wx_exec_data`
Extra	Opzionali	Programmi e test privati, test lato host, funzioni in più	Non devono cambiare comandi, output, formati e test

1.6 Formati e messaggi esatti

Questa sezione raccoglie i dettagli che devono essere identici in tutte le squadre, perché il test kit comune li controlla byte per byte.

Salvo indicazione diversa valgono tre convenzioni: ogni numero di più di un byte in IMG1 e SFT1 è un intero senza segno a 32 bit little-endian; gli offset si contano dal primo byte dell'immagine o del file; un intervallo scritto `[start, end)` contiene `start` ma non `end`.

1.6.1 L'immagine IMG1

Dove si trova. QEMU carica il file `build/fs.img` senza modificarlo, a partire dall'indirizzo FLASH `0x00200000` (l'opzione esatta è in). L'immagine può occupare

al massimo 2 MiB, cioè `0x00200000..0x003fffff`. Il kernel la legge lì, senza scriverla e senza copiarla.

Header. L'immagine inizia con un header di esattamente 20 byte:

Byte	Dim.	Campo	Valore
`0..3`	4	`magic`	magic number dell'immagine: byte `49 4d 47 31` (`IMG1`), valore numerico `0x31474d49`
`4..7`	4	`version`	esattamente `1`
`8..11`	4	`image_size`	dimensione totale dell'immagine, da 20 a `0x00200000` byte
`12..15`	4	`count`	numero di voci della directory, da 1 a 32
`16..19`	4	`directory_offset`	dove inizia la directory; multiplo di 4, almeno 20

Il valore `directory_offset + count * 24` non deve andare in overflow e non deve superare `image_size`. Il packer della squadra mette sempre la directory al byte 20.

Directory. La directory ha esattamente `count` voci da 24 byte:

Byte	Dim.	Campo	Valore
`0..15`	16	`name`	nome del programma, seguito da un byte NUL
`16..19`	4	`offset`	dove inizia il file programma; multiplo di 4
`20..23`	4	`size`	dimensione esatta del file programma, almeno 40 byte

Un nome ha da 1 a 15 caratteri, scelti tra `A`–`Z`, `a`–`z`, `0`–`9`, `_` e `-`. Due voci non possono avere lo stesso nome. Il kernel ignora i byte dopo il primo NUL; il packer li mette a zero e rifiuta i nomi duplicati.

File programma. `contents_start` è la fine della directory, arrotondata a un multiplo di 4. Ogni file inizia a `contents_start` o dopo, ogni intervallo `[offset, offset + size)` sta dentro `image_size` e due file non si sovrappongono mai. Le voci possono essere in qualunque ordine, tra un file e l'altro ci possono essere buchi e le dimensioni dei file non devono per forza essere multipli di 4.

Che cosa fa il packer. Mantiene i file nell'ordine della riga di comando, allinea l'inizio di ciascuno con padding di zeri e pone `image_size` uguale alla fine dell'ultimo file. Rifiuta zero file, più di 32 file e immagini più grandi di 2 MiB.

Che cosa fa il kernel con un'immagine sbagliata. Se anche una sola di queste regole è violata, l'intera immagine non è valida, e allora:

- `ls` stampa `error: no valid image`;
- `run <name>` stampa `error: program cannot be loaded`;

- la shell continua a funzionare.

Un'immagine valida può comunque contenere un file programma rotto: `ls` lo elenca, e `run` trova il problema quando controlla il file.

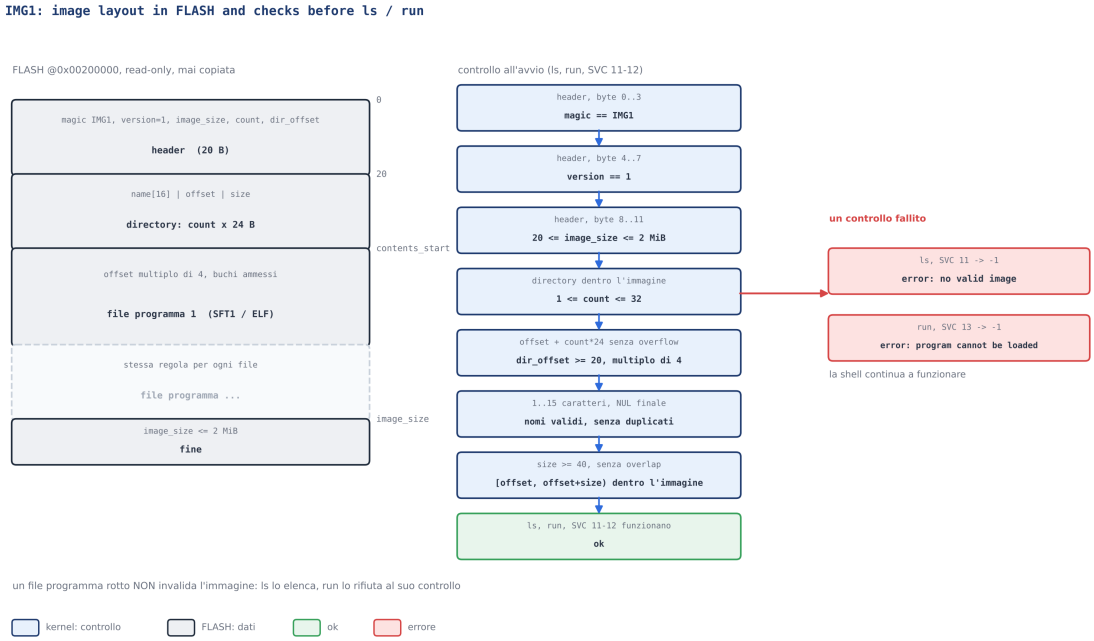


Figura 1.4: A sinistra il layout dell'immagine IMG1 in FLASH; a destra la catena di controlli che il kernel esegue prima di `ls` e `run`. Un controllo fallito rende invalida l'intera immagine, ma non cambia nessuno slot.

1.6.2 Il file programma SFT1 (solo small)

Le squadre small salvano i programmi in file SFT1. Ogni file inizia con un header di esattamente 40 byte:

Byte	Dim.	Campo	Valore
`0..3`	4	`magic`	magic number di un file SFT1: byte `53 46 54 31` (`SFT1`), valore numerico `0x31544653`
`4..7`	4	`version`	esattamente `1`
`8..11`	4	`header_size`	esattamente 40
`12..15`	4	`texttt_size`	dimensione di codice e costanti; diversa da zero, multipla di 4
`16..19`	4	`data_size`	dimensione dei dati inizializzati; multipla di 4
`20..23`	4	`bss_size`	dimensione del BSS; multipla di 4; questi byte non sono nel file
`24..27`	4	`entry`	entry point, contato dall'inizio del codice; il bit 0 vale 1 (Thumb) e `entry & ~1` è minore di `texttt_size`
`28..31`	4	`reloc_offset`	inizio della relocation list; multiplo di 4 ed esattamente `40 + texttt_size + data_size`
`32..35`	4	`reloc_count`	numero di voci della relocation list
`36..39`	4	`file_size`	dimensione dell'intero file; uguale alla `size` della voce di directory e alla fine della relocation list

Layout. Codice e costanti occupano `[40, 40 + texttt_size)`. I `data_size` byte successivi contengono i dati inizializzati. Poi viene la relocation list: `reloc_count` parole, ognuna delle quali è l'offset di una parola da correggere. Quindi `file_size = reloc_offset + 4 * reloc_count`, e dopo la lista non c'è nulla.

Da chiarire con il professore: da dove si contano gli offset della relocation list. La regola generale () dice che gli offset si contano dal primo byte del file, e per queste voci il .md non indica eccezioni, mentre per `entry` dice esplicitamente che si conta dall'inizio del codice. Conviene confermarlo, anche perché eventuali file SFT1 rotti generati dal test kit vanno letti nello stesso modo.

Limiti. `texttt_size + data_size + bss_size` non deve andare in overflow e non deve superare 3072 byte. `reloc_count` può essere zero ed è al massimo `(texttt_size + data_size) / 4`. Ogni offset di relocation è multiplo di 4, compare una sola volta e indica una parola intera di 4 byte dentro il codice o i dati.

Come lo usa il loader. Prima di scrivere qualunque cosa nello slot, il loader controlla ogni campo, intervallo, calcolo e relocation, compreso ogni valore `slot_base + original_word`, che non deve andare in overflow sui 32 bit. Poi copia codice e dati, riempie di zeri il BSS e somma la base dello slot a ogni parola elencata. La parola originale può puntare fuori dal programma. Sommare la base mantiene il bit 0 di un puntatore a funzione Thumb.

Errori. Qualunque errore SFT1 dà il risultato interno `-4` e il messaggio `error: program cannot be loaded`. Nessun byte del programma viene copiato o eseguito.

1.6.3 I file ELF dei programmi e l'host tool

Com'è fatto ogni ELF. I programmi sono eseguibili Thumb freestanding per ARM EABI Cortex-M3, non oggetti del kernel. Seguono la normale convenzione di chiamata del C, senza libreria C e senza heap. Ogni ELF è un file ARM a 32 bit, little-endian, versione 1, di tipo `ET_EXEC`, linkato all'indirizzo zero con `-emit-relocs`. I suoi record hanno le dimensioni standard: header del file da 52 byte, program header da 32 byte, section header da 40 byte, simboli da 16 byte e record `REL` da 8 byte. Almeno un segmento `PT_LOAD` descrive la parte da caricare. L'entry point è un indirizzo Thumb dentro il codice. L'ELF non deve contenere record `RELA`, dynamic linking, simboli non definiti, né tipi di relocation diversi da quelli accettati qui sotto per il contenuto caricato. I dettagli del Makefile li decide la squadra.

Quali relocation contano. Il campo `sh_info` di ogni sezione di relocation dice a quale sezione si applica. Contano solo le relocation delle sezioni caricate in memoria (le sezioni allocated); quelle delle altre sezioni, come le informazioni di debug (compreso `R_ARM_NONE`), si ignorano. Per il contenuto caricato:

- `R_ARM_ABS32` (un indirizzo completo a 32 bit) va corretta sommando la base dello slot;
- `R_ARM_THM_CALL` e `R_ARM_THM_JUMP24` (salti Thumb) sono relative alla posizione corrente: si controllano ma non si modificano;
- ogni altro tipo viene rifiutato.

Small: conversione da ELF a SFT1. Il converter legge header ELF da 52 byte, program header da 32 byte e section header da 40 byte. Tutte le tabelle devono essere valide e stare dentro il file, e deve esserci almeno un `PT_LOAD`. L'indirizzo di caricamento più basso e il contenuto caricato partono da zero. Le sezioni caricate non si sovrappongono e stanno dentro i segmenti caricabili. Le sezioni con contenuto iniziale vengono prima di tutte le sezioni `SHT_NOBITS` (BSS), e le sezioni read-only prima di quelle scrivibili.

Il converter riempie di zeri i buchi e il padding di allineamento, arrotonda a un multiplo di 4 la fine del codice, dei dati inizializzati e della memoria totale, e rifiuta i programmi più grandi di 3072 byte. Accetta solo record `REL` validi che usano simboli definiti. I site `R_ARM_ABS32` allineati a parola diventano la relocation list SFT1, ordinata e senza duplicati. I site `R_ARM_THM_CALL` e `R_ARM_THM_JUMP24` allineati a mezza parola si controllano e non producono voci. Il converter rifiuta `RELA`, relocation nel BSS, simboli non definiti, site non validi e ogni altro tipo di relocation per il contenuto caricato.

Small: comandi dell'host tool. I comandi sono esattamente:

```
python3 tools/mkimage.py flat PROGRAM.elf PROGRAM.sft
python3 tools/mkimage.py image build/fs.img NAME=PROGRAM.elf [NAME=PROGRAM
    .elf ...]
```

In caso di successo `flat` stampa `SFT1: ELF -> OUTPUT (N bytes)` e `image` stampa `IMG1: OUTPUT (N bytes, C programs)`. Nella small `image` riceve file ELF, ma

il kernel small legge solo SFT1: nell'immagine finiscono quindi i programmi convertiti in SFT1.

Prima di scrivere qualunque output, lo strumento rifiuta argomenti, nomi e file ELF sbagliati e risultati troppo grandi. Gli errori passano da `argparse` di Python, che scrive `mkimage.py: error: <reason>` su standard error ed esce con stato 2.

Big: comando dell'host tool. Il comando è esattamente:

```
python3 tools/mkimage.py image build/fs.img NAME=PROGRAM.elf [NAME=PROGRAM
    .elf ...]
```

Valgono tutte le regole IMG1. Lo strumento copia ogni ELF byte per byte, senza modifiche. Le squadre big non hanno bisogno del comando `flat`.

Big: che cosa controlla il kernel in un ELF. Il kernel controlla l'identità dell'ELF, le dimensioni dei record e ogni tabella di program header o di sezioni che usa. Controlla i segmenti caricabili e le sezioni caricate che gli servono per trovare i target delle relocation, tutte le sezioni `REL` e i simboli usati dalle relocation che contano. Prima di usare un valore, verifica che ogni calcolo, tabella, record, segmento, sezione, site di relocation e intervallo copiato stia dentro il file e dentro lo slot. Non deve controllare i nomi delle sezioni, il contenuto di debug o delle stringhe, né i campi dei simboli che non usa.

Per le sezioni caricate applica `R_ARM_ABS32` sommando la base dello slot e accetta senza modifiche `R_ARM_THM_CALL` e `R_ARM_THM_JUMP24` risolte. Ignora le relocation delle altre sezioni, purché i loro record stiano dentro il file. Per qualunque altro tipo in una sezione caricata stampa prima la riga

```
loader: unsupported relocation type <type>
```

con il tipo come numero decimale, e poi il normale messaggio di rifiuto. Un ELF malformato viene rifiutato senza fault del kernel.

1.6.4 Mappa di memoria e slot

Si usa la mappa di memoria completa del linker script della base:

Area	Indirizzi
FLASH della board (4 MiB), usata dal kernel	da `0x00000000`
Immagine `fs.img`	da `0x00200000`, mai oltre `0x00400000`
Partizione RAM della shell	`0x20000000..0x20000fff`
Partizione dell'idle task	`0x20001000..0x20001fff`
Slot 0	`0x20002000..0x20002fff`
Slot 1	`0x20003000..0x20003fff`
Slot 2	`0x20004000..0x20004fff`
Slot 3	`0x20005000..0x20005fff`

Dati e stack del kernel stanno fuori dalle partizioni di shell e idle task. Gli slot sono quattro, fissi, da 4096 byte, ciascuno allineato a 4096 byte.

Scelta dello slot. Il loader prende sempre il primo slot libero, nell'ordine 0, 1, 2, 3.

Dentro uno slot. Codice, dati, BSS e stack condividono lo slot:

Offset nello slot	Contenuto
da 0 a 3071	il programma (codice + dati + BSS), che usa al massimo i 3072 byte bassi
da 3072 a 4095	lo stack (1024 byte), che cresce verso il basso; i frame iniziali preparati dal codice dato usano i 64 byte in cima

Lo stack pointer iniziale è la fine dello slot.

Esecuzione e protezione. I programmi girano unprivileged in Thread mode, sul process stack (PSP), sotto lo scheduler round-robin preemptive già esistente. Non devono accedere alla RAM del kernel né ad altri slot. Come nel setup MPU della base, la FLASH `0x00000000..0x003fffff` si può leggere ed eseguire da ogni task unprivileged: serve alla shell, che ha il codice in FLASH, mentre un programma caricato esegue solo la copia nel suo slot.

- Small: una regione MPU dà lettura, scrittura ed esecuzione sullo slot corrente.

- Big: le regioni danno al codice lettura ed esecuzione, senza scrittura, e a dati e stack lettura e scrittura, senza esecuzione (W^X).

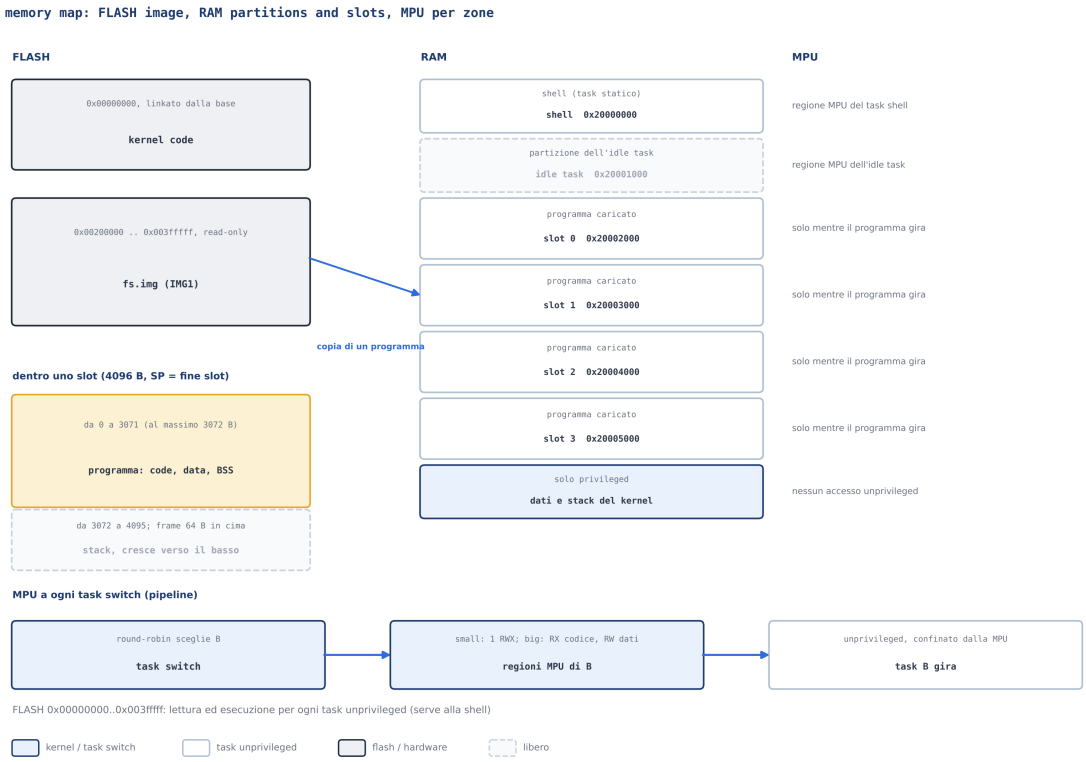


Figura 1.5: Mappa di memoria: FLASH con kernel e immagine, RAM con partizioni, slot e dati del kernel, e a destra la regione MPU di ciascuna zona. In uno slot il programma usa i primi 3072 byte, lo stack i restanti 1024; a ogni task switch la MPU viene riprogrammata.

1.6.5 System call

Chiamate già esistenti. I programmi possono linkare gli stub di `lib/syscalls.s`. I prototipi sono:

```
int sys_write(const char *buf, unsigned len);
int sys_read(void);
uint32_t sys_uptime(void);
void sys_exit(void);
void task_yield(void);
void task_delay(uint32_t ticks);
int task_delay_until(uint32_t *last_wake,
                     uint32_t period);
void trace_dump(void);
uint32_t task_tick_count(void);
```

SVC	Funzione	Risultato
1	`sys_write`	numero di byte accettati, `0..len`; `-1` se l'intervallo di byte non sta tutto nella partizione del chiamante o in FLASH
2	`sys_read`	prossimo byte della UART, `0..255`, oppure `-1` se non c'è nessun byte in attesa
3	`sys_uptime`	millisecondi dall'avvio del clock
4	`sys_exit`	non ritorna; per i programmi caricati vedi sotto
5	servizio di avvio, solo kernel	un task riceve `-1`
6	`task_yield`	chiede un task switch; il chiamante resta pronto a girare
7	`task_delay`	blocca per il numero indicato di tick da 10 ms; zero equivale a una yield
8	`task_delay_until`	`0`; `-1` se la destinazione allineata di 4 byte non sta tutta nella partizione del chiamante
9	`trace_dump`	stampa la trace esistente dello scheduler
10	`task_tick_count`	tick dall'avvio dello scheduler

La `lib/printf.h` fornita supporta `%c`, `%s`, `%d`, `%u`, `%x` e `%%`, e restituisce il numero di caratteri stampati.

Nuove chiamate del loader. Gli argomenti passano nei registri `r0`, `r1` e `r2` del chiamante e il risultato torna in `r0`. Ogni intervallo di memoria che appartiene al chiamante deve stare tutto nella sua partizione.

```
int sys_image_count(void);
int sys_image_name(unsigned index, char *name,
                   unsigned capacity);
int sys_program_run(const char *name);
int sys_program_wait(int task_id);
```

SVC	Nome	Risultato
11	`SYS_IMAGE_COUNT` `sys_image_count`	numero di voci della directory, `1..32`; `-1` se l'immagine non è valida
12	`SYS_IMAGE_NAME` `sys_image_name`	copia il nome completo, con il NUL finale, e restituisce `0`; `-1` se l'immagine o l'indice non sono validi, se `capacity` non è in `1..16`, se la destinazione è fuori dalla partizione del chiamante o se il nome non ci sta
13	`SYS_PROGRAM_RUN` `sys_program_run`	il task id (un numero non negativo); `-1` per un puntatore o una stringa del nome non validi o per un'immagine non valida; `-2` se il programma non c'è; `-3` se nessuno slot è libero; `-4` se il file programma non è valido
14	`SYS_PROGRAM_WAIT` `sys_program_wait`	aspetta la fine di quel programma, oppure restituisce subito lo stato salvato quando è finito: `0` dopo return o exit, `-2` dopo un fault recuperato; `-1` per zero, per l'id del chiamante stesso o per un id malformato, sconosciuto o vecchio

Altri dettagli:

- *SVC 12.* La FLASH non è una destinazione valida. Se `capacity` è troppo piccola, la chiamata copia la prima parte del nome, comunque terminata da NUL, e restituisce `-1`.
- *SVC 13.* `name` deve essere non vuoto e terminare con NUL entro 16 byte, quindi ha al massimo 15 caratteri. Ogni byte letto deve stare nella partizione del chiamante o in FLASH `0x00000000..0x003fffff`. I caratteri non vengono confrontati con le regole dei nomi: un nome che non può esistere restituisce semplicemente `-2`.
- *SVC 14.* Non c'è un puntatore di output: quando il programma finisce, il risultato viene scritto nel valore di `r0` salvato del chiamante in attesa. Un task id contiene il numero dello slot nei bit 1:0 e, sopra, un contatore di generation a 28 bit diverso da zero: i primi id sono 4, 5, 6 e 7 per gli slot 0, 1, 2 e 3, e il programma successivo nello slot 0 riceve l'id 8 (in pratica $id = 4 \cdot generation + slot$). Ogni slot conserva uno stato di fine finché non termina il suo programma successivo. Aspettare non riserva uno slot liberato. Più task possono aspettare lo stesso programma in esecuzione.
- *SVC 4.* Non ha argomenti e non ritorna. Per un programma caricato equivale a tornare da `main`: stampa il messaggio di uscita, salva lo stato 0, sveglia i task in attesa, libera lo slot e passa a un altro task. Per un task statico termina il task, senza messaggio di programma e senza uno slot riusabile.
- *Chiamate sconosciute.* I numeri di SVC sconosciuti, la SVC 0 e un task che chiama la SVC 5 (riservata al kernel) ricevono `-1`.
- *SVC 1 con lunghezza 0.* Restituisce 0 senza leggere memoria se il puntatore è dentro la partizione del chiamante o la FLASH, o esattamente sul loro bordo; qualunque altro puntatore dà `-1`. I calcoli sugli intervalli non devono andare in overflow.

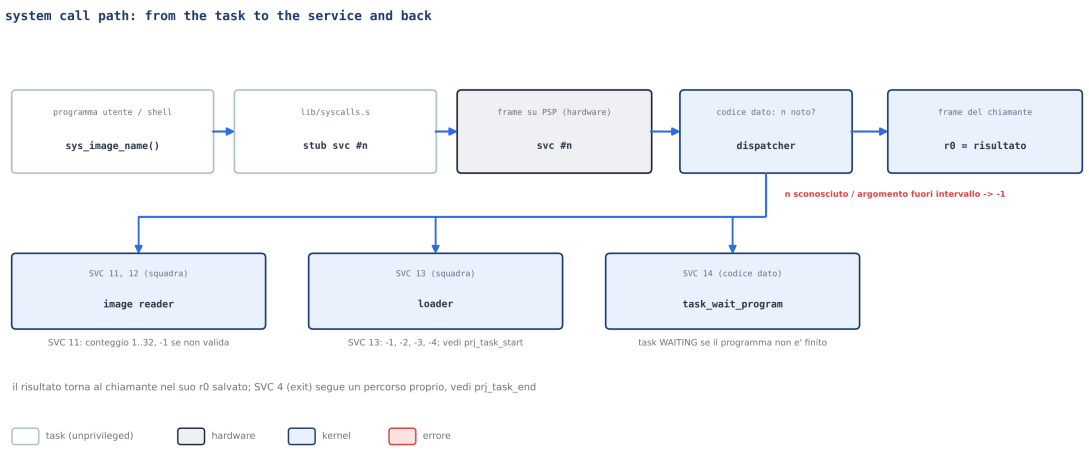


Figura 1.6: Percorso di una system call: lo stub in `lib/syscalls.s` esegue `svc #n`, il dispatcher controlla numero e argomenti, e il risultato torna in `r0` del chiamante. Le SVC 11, 12 e 13 sono servizi del progetto; la SVC 14 è già data.

1.6.6 Input e output della shell

All'avvio. Il kernel stampa:

```
scratch-os: kernel started
scheduler: round robin, quantum 5 ticks
```

Lettura di una riga. La shell stampa il prompt `scratch>` , con uno spazio finale e senza andare a capo. Fa l'eco di ogni carattere memorizzato, senza cambiarlo. Un CR o un LF stampa un solo a capo e chiude la riga. La shell memorizza e ripete in eco al massimo 31 caratteri: quelli in più vengono scartati, senza eco, fino alla fine della riga. Non c'è line editing e gli spazi non vengono tolti.

Comandi. Gli unici comandi sono esattamente `help`, `ls` e `run` seguito da un nome non vuoto. Un secondo spazio dopo `run` fa parte del nome. Una riga vuota, `run` da solo, caratteri in più dopo `help` o `ls`, e qualunque altro input stampano `error: unknown command`.

`help` stampa esattamente:

```
help - show commands
ls - list programs
run <name> - run a program
```

`ls` stampa i nomi dei programmi nell'ordine della directory, uno per riga. Se l'immagine non è valida stampa:

```
error: no valid image
```

Quando `run <name>` fallisce, la shell stampa, rispettivamente per i risultati `-2`, `-3` e per qualunque altro valore negativo:

```
error: program not found
error: no free slot
error: program cannot be loaded
```

Quando `run <name>` riesce, il kernel stampa la riga seguente, con numeri esadecimali minuscoli e senza zeri iniziali. A stamparla è la funzione data `task_program_create`:

```
loader: <name> slot=<slot> base=0x<base> id=<id>
```

Mentre il programma gira, la shell aspetta e non stampa il prompt. Quando il programma torna da `main` o chiama la SVC 4, il sistema stampa:

```
program <name> exited
```

Quando un programma caricato provoca un fault recuperabile, il sistema stampa:

```
<kind> in loaded program: CFSR=0x<cfsr>[ address=0x<mmfar>]
program <name> faulted
```

`<kind>` è esattamente `MemManage`, `BusFault`, `UsageFault` o `HardFault`. La parte tra parentesi quadre compare solo quando è attivo il bit `MMARVALID`. La wait restituisce `-2`, lo slot viene liberato e il prompt ricompare.

Alcuni fault non sono recuperabili: un MemManage o un BusFault che avviene mentre il processore salva o ripristina lo stack frame, un HardFault causato da un errore di stack di questo tipo, e qualunque fault fuori da un programma caricato. In questi casi il sistema stampa la riga seguente e si ferma, senza un altro prompt:

`<kind>: unrecoverable CFSR=0x<cfsr> HFSR=0x<hfsr>`

Il prompt ricompare dopo ogni altro comando, ogni caricamento fallito e ogni fault recuperato. Comandi extra o output di debug non devono mai comparire in queste trascrizioni, né ritardarle o cambiarle.

Due conseguenze delle regole.

- Sulla riga stanno 31 caratteri, quindi dopo `run` un nome può arrivare a 27 caratteri, ma la SVC 13 ne accetta al massimo 15. Con un nome più lungo la SVC 13 restituisce `-1` e la shell stampa `error: program cannot be loaded`, non `error: program not found`.
- Con due spazi dopo `run` il nome cercato inizia con uno spazio, quindi non esiste: la SVC 13 restituisce `-2` e la shell stampa `error: program not found`.

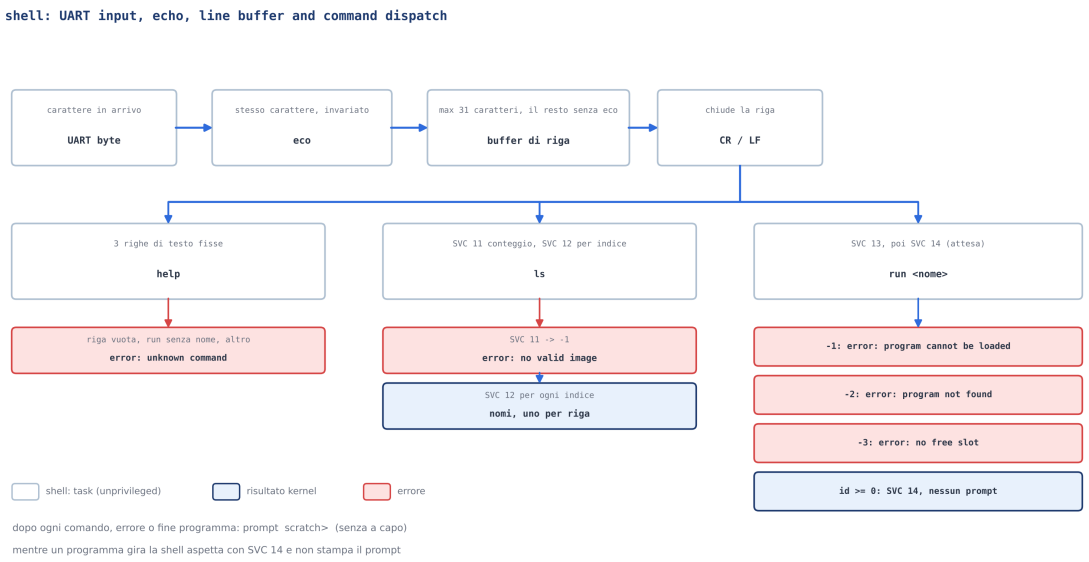


Figura 1.7: Percorso dell'input della shell: ogni carattere viene ripetuto in eco, la riga si chiude con CR o LF e il comando decide quali system call usare. Dopo ogni esito la shell torna al prompt, tranne mentre un programma gira.

1.6.7 Il codice dato da usare

La base contiene già quattro task record (TCB) riusabili per i programmi caricati, i task id con il contatore di generation, i record di attesa e di stato di fine, e gli stack frame iniziali. Gestisce anche return, exit e fault recuperati, sveglia i task in attesa e implementa la SVC 14. Questo codice non va sostituito né copiato.

Il codice della squadra fornisce la shell tramite `task_project_configure()` e `task_configure_shell()`. Fornisce anche le SVC da 11 a 13, i controlli sui file, il caricamento, la relocation, i controlli di entry e stack e i permessi MPU. La funzione data `task_program_create` si chiama solo dopo che il programma è stato caricato per intero e senza errori.

L'interfaccia è dichiarata in `kernel/task.h`:

```
task_project_configure(void)
task_configure_shell(name, entry, stack_top,
                     part_start, part_end,
                     data_start, data_end, data_load)
task_program_slot_free(slot)
task_program_create(slot, name, entry, stack_top,
                    executable)
task_wait_program(id, saved_result)
```

Funzione	Che cosa fa e come si usa
`task_project_configure`	hook weak della base: la squadra ne scrive una definizione normale (strong) e lì configura la shell
`task_configure_shell`	imposta il task della shell; si passano gli indirizzi dei simboli del linker della shell corrispondenti
`task_program_slot_free`	dice se uno slot è libero; è falsa per uno slot occupato o inesistente
`task_program_create`	crea il task e restituisce il suo id, oppure `-1`; copia al massimo 15 caratteri del nome, costruisce i frame iniziali da 64 byte, mette il task in READY e stampa la riga `loader:` esatta
`task_wait_program` e codice della SVC 14	supporto per l'attesa e lo stato di fine; non si aggiunge un altro modo di aspettare attraverso il suo helper del kernel
`task_finish_current`, `task_fault_current`, `task_current_is_program`, entry dei fault e dispatcher	supporto per l'exit e il recupero dai fault; non va sostituito né copiato

Simboli del linker. Il linker esporta, come indirizzi:

Shell	Slot e immagine
`_shell_part_start`	`__program_slots_start`
`_shell_stack_top`	`__program_slot_N_start` (per ogni slot)
`_shell_part_end`	`__program_slot_N_end` (per ogni slot)
`_shell_data_start`	`__program_slots_end`
`_shell_data_end`	`__program_slot_size`
`_shell_data_load`	`__program_slot_count`
	`__fs_image_start`

Le sezioni scrivibili della shell vengono da `shell.o`; con un altro nome di oggetto servono regole del linker equivalenti. L'entry del codice della shell e i suoi dati scrivibili devono restare dentro la sua partizione di 4 KiB. *Da chiarire con il professore:* questa frase sembra in contrasto con R7, dove il codice della shell sta in FLASH, mentre la partizione della shell è in RAM (`0x20000000..0x20000fff`).

Estensioni per la big. Le squadre big possono aggiungere campi ai task record ed estendere il setup MPU fatto a ogni task switch, così che codice e dati abbiano permessi diversi. Il comportamento dato deve restare lo stesso. L'argomento `executable` serve alla politica "slot intero" della small: da solo non dà W^X.

1.7 Come funziona il sistema finito

Ecco il percorso completo, dalla build alla fine di un programma. I passi descrivono il comportamento richiesto; come dividerlo in funzioni lo decide la squadra.

1.7.1 Build sul PC

1. Ogni sorgente in `programs/` viene compilato e linkato da solo, all'indirizzo `0` e con `-emit-relocs`: si ottiene un ELF per programma.

2. `tools/mkimage.py image` impacchetta i programmi in `build/fs.img` (IMG1), nell'ordine della riga di comando: nella small come file SFT1, nella big come ELF invariati.

3. Il kernel, con la shell, diventa `build/scratch-os.axf`. QEMU (`mps2-an385`) esegue il kernel dalla FLASH a partire da `0x00000000` e carica `build/fs.img` a `0x00200000`.

1.7.2 Avvio

4. Il kernel stampa le due righe di boot ().

5. La base chiama l'hook `task_project_configure()`, nel quale la squadra chiama `task_configure_shell(...)` con i simboli del linker della shell: la shell diventa un task statico con la partizione `0x20000000..0x20000fff`.

6. Lo scheduler round-robin (quantum di 5 tick) comincia a far girare i task, tra cui la shell.

1.7.3 Shell e comandi

7. La shell stampa `scratch>` , legge i caratteri dalla UART, ne fa l'eco e chiude la riga con CR o LF.

8. Per `ls` usa la SVC 11 (quante voci ci sono) e poi la SVC 12 per ogni indice (il nome).

9. Per `run <name>` usa la SVC 13.

1.7.4 Caricamento (SVC 13)

10. Il kernel controlla il nome (puntatore e stringa) e l'immagine: se qualcosa non va restituisce `-1`.

11. Cerca il nome nella directory: se non c'è, restituisce `-2`.

12. Cerca il primo slot libero con `task_program_slot_free`: se sono tutti occupati, restituisce `-3`.

13. Controlla tutto il file programma (campi, intervalli, calcoli, relocation, entry e stack) senza scrivere ancora nello slot: al primo errore restituisce `-4`. Nella big, un tipo di relocation non supportato fa stampare prima la riga `loader: unsupported relocation type <type>`.

14. Solo a questo punto scrive nello slot: copia codice e dati, azzera il BSS e somma la base dello slot a ogni parola da correggere.

15. Chiama `task_program_create(...)`: il codice dato prepara i frame iniziali da 64 byte, mette il task in READY e stampa la riga `loader:`. La SVC 13 restituisce il task id.

Un caricamento fallito, o il tentativo di avviare un quinto programma mentre ne girano già quattro, non deve cambiare nessuno slot.

Da chiarire con il professore: il .md elenca i codici d'errore della SVC 13 ma non dice quale prevale quando ce n'è più di uno (per esempio file non valido e nessuno slot libero). L'ordine dei passi 10–13 è quindi una proposta; si noti che il controllo di `slot_base + original_word` richiede di conoscere già lo slot.

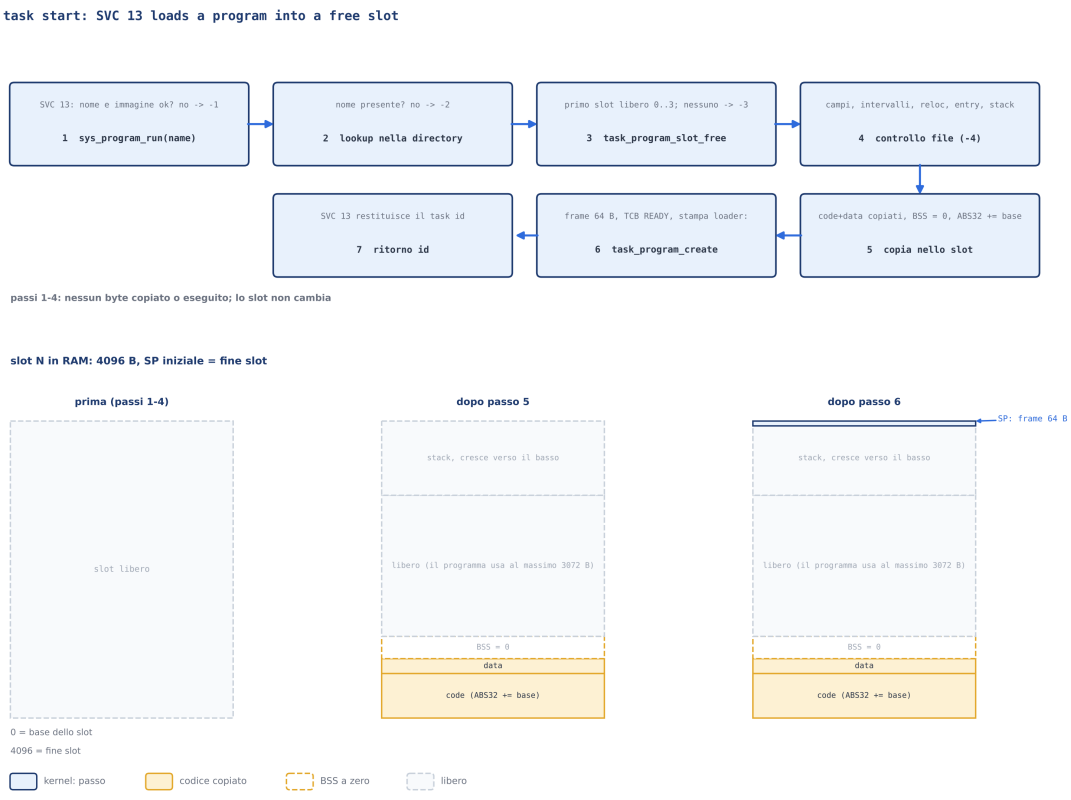


Figura 1.8: Caricamento di un programma con SVC 13: i passi 1–4 controllano tutto senza scrivere nello slot, dal passo 5 la copia riempie lo slot e il passo 6 crea il task. In basso, lo stato dello slot dopo il passo 5 e dopo il passo 6.

1.7.5 Esecuzione

16. La shell chiama la SVC 14 con l'id ricevuto e passa nello stato WAITING. Il programma può anche partire prima che la shell chiami la SVC 14.

17. Il programma gira unprivileged in Thread mode, sul PSP, con codice e dati nel suo slot; la MPU, programmata a ogni task switch, lo confina nello slot. `_start` chiama `main`.

1.7.6 Fine del programma

18. Ritorno da `main` o `sys_exit()`: il sistema stampa `program <name> exited` e lo stato di fine è 0.

19. Fault recuperabile: il sistema stampa la riga del fault e `program <name> faulted`; lo stato di fine è `-2`.

20. In entrambi i casi il codice dato salva lo stato, mette direttamente il task record in UNUSED, sveglia i task in attesa e chiede un task switch: lo slot è subito riusabile.

21. La SVC 14 della shell restituisce lo stato, che la shell ignora perché il kernel ha già stampato come è finito il programma, e la shell stampa di nuovo il prompt.

22. Fault non recuperabile: il sistema stampa la riga `unrecoverable` e si ferma, senza prompt.

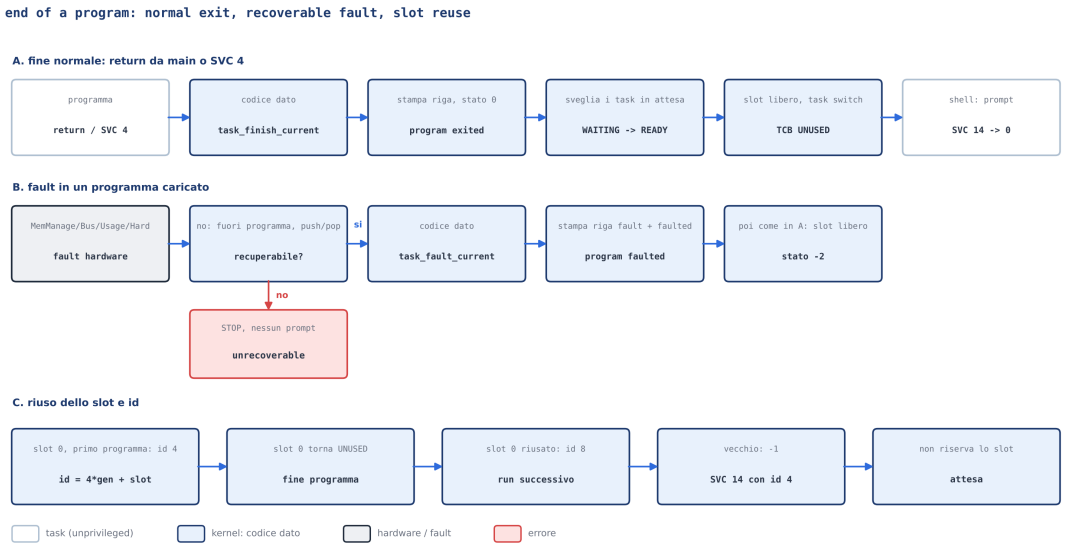


Figura 1.9: Fine di un programma: uscita normale, fault recuperabile e riuso dello slot. Il codice dato libera lo slot subito e riporta il prompt; un fault non recuperabile ferma invece il sistema.

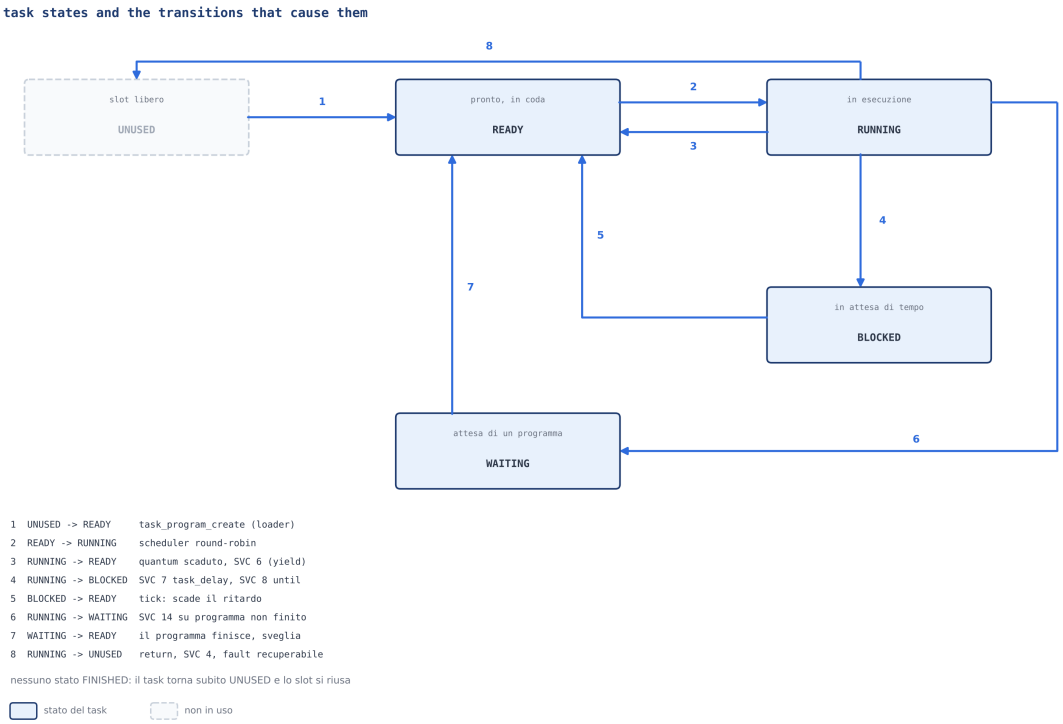


Figura 1.10: Stati dei task e transizioni, con la funzione o la SVC che le causa. Il progetto non introduce uno stato FINISHED: un programma terminato rende lo slot subito `UNUSED`.

1.8 Vincoli e divieti

1.8.1 Che cosa non si può fare

- Sostituire o copiare il codice dato del ciclo di vita dei programmi e del recupero dai fault: deve restare lui al comando.
- Aggiungere un altro modo di aspettare un programma attraverso l'helper del kernel di `task_wait_program`.
- Scrivere nell'immagine o copiarla intera in RAM.
- Copiare o eseguire un qualunque byte del programma prima di aver finito tutti i controlli. Un caricamento fallito e il tentativo di avviare un quinto programma mentre ne girano quattro non devono cambiare nessuno slot.
- Chiamare `task_program_create` prima che il programma sia caricato per intero e senza errori.
- Stampare comandi extra o output di debug: non devono mai comparire nelle trascrizioni, né ritardarle o cambiarle.
- Far condividere codice tra programmi e kernel, o usare nei programmi la libreria C o un heap.

- Produrre ELF con record `RELA`, dynamic linking, simboli non definiti o altri tipi di relocation per il contenuto caricato.

- Rendere recuperabili i fault durante il salvataggio dello stack frame, i fault nel kernel e quelli fuori dai programmi caricati: devono restare non recuperabili.

- Cambiare, con programmi o test extra della squadra, i comandi, l'output, i formati o il comportamento dei test fissati.

- Nella big, lasciare una regione MPU sovrapposta con lettura, scrittura ed esecuzione: romperebbe W^X.

1.8.2 Che cosa si può cambiare

- Si possono aggiungere moduli e aggiungere campi al codice delle system call, dei task e della MPU.

- Le squadre big possono aggiungere campi ai task record ed estendere il setup MPU di ogni task switch, purché il comportamento dato resti lo stesso.

- La ricetta `make image` della base va cambiata ().

1.8.3 Che cosa è fisso e che cosa decide la squadra

Fisso per tutte le squadre	Deciso dalla squadra
Board e CPU di QEMU, indirizzo dell'immagine e limite di dimensione, formato IMG1 e suoi limiti	Organizzazione di moduli e file
Versione assegnata in base alla dimensione della squadra	Organizzazione e coordinamento del lavoro tra i membri
Indirizzi, dimensione e allineamento degli slot, scelta del primo libero, limite di 3072 byte per il programma, stack di 1024 byte	Funzioni di supporto per i controlli e strutture dati interne, purché calcoli e intervalli siano sicuri
Numeri, argomenti e risultati delle system call, regole sui puntatori, task id, ciclo di vita dei programmi	Nomi delle funzioni del kernel, tranne i punti di integrazione dati
Comandi, prompt, output, limite dell'input e messaggi d'errore della shell	Struttura della shell e modo in cui bufferizza l'input
Regole di build dei programmi: link all'indirizzo zero, entry Thumb, `-emit-relocs`, ritorno da `main` che vale come exit	Programmi di prova privati in più e test lato host
Small: formato SFT1, regole di conversione e di relocation, una regione MPU con lettura, scrittura ed esecuzione	Struttura del lettore ELF lato host (non serve nessuna libreria esterna)
Big: ELF salvato invariato, sottoinsieme ELF richiesto, relocation e controlli, messaggio con il tipo numerico	Struttura del lettore ELF nel kernel; il contenuto non usato non va controllato
Big: codice lettura + esecuzione; dati e stack lettura + scrittura, non eseguibili	Quali regioni MPU danno questi permessi
Codice e chiamate dati del ciclo di vita	Stile della documentazione e della presentazione
Test suite e trascrizioni, punti, penalità	Funzioni extra che non cambiano nessun comportamento fisso

1.8.4 Regole di stile e di codice

Il .md non impone uno stile di scrittura del codice. Le regole pratiche vengono dal criterio di efficienza e dai controlli richiesti:

- uso delle risorse limitato (bounded);
- nessuna copia inutile dell'immagine e nessuna allocazione dinamica;
- controlli chiari e ben strutturati, senza duplicazioni evitabili;
- ogni calcolo su offset, dimensioni e intervalli protetto dall'overflow;
- per leggere gli ELF lato host non serve nessuna libreria esterna.

1.9 Comandi

Il .md riporta solo i comandi seguenti. Il comando completo di build (per esempio il target del Makefile che crea programmi e immagine) lo sceglie la squadra e va scritto nella documentazione.

1.9.1 Build ed esecuzione

`make run` compila la base e la avvia in QEMU, usando un'immagine già esistente; `make image` è la ricetta della base per l'immagine, da cambiare.

```
# Compila e avvia in QEMU; usa un'immagine esistente
make run
# Ricetta della base per l'immagine: da cambiare
make image
```

Questa è l'opzione con cui QEMU carica l'immagine nella FLASH, senza modificarla, a partire dall'indirizzo `0x00200000`:

```
-device loader,file=build/fs.img,addr=0x00200000,force-raw=on
```

1.9.2 Host tool

`flat` (solo small) converte un ELF in SFT1; `image` crea `build/fs.img` dai programmi elencati, nell'ordine dato.

```
# Small: converte un ELF in un file SFT1
python3 tools/mkimage.py flat PROGRAM.elf PROGRAM.sft
# Small e big: crea build/fs.img (IMG1)
python3 tools/mkimage.py image build/fs.img NAME=PROGRAM.elf [NAME=PROGRAM
    .elf ...]
```

1.9.3 Test

Il runner esegue la suite small (10 test) o big (14 test).

```
# Suite small
python3 run_tests.py --suite small --kernel KERNEL.axf --image fs.img
# Suite big
python3 run_tests.py --suite big --kernel KERNEL.axf --image fs.img
```

`KERNEL.axf` e `fs.img` stanno al posto dei percorsi veri: il runner usa come kernel `build/scratch-os.axf`, e l'immagine della squadra è `build/fs.img`. Il runner si lancia dalla cartella del test kit, oppure da un'altra cartella con i percorsi giusti di kernel e immagine.

Opzione	Effetto
`-list`	elenca i test
`-test NAME`	esegue solo il test indicato; si può ripetere
`-startup-timeout`	limite di tempo per l'avvio (default 5 secondi)
`-test-timeout`	limite di tempo per ogni test (default 5 secondi)
`-global-timeout`	limite di tempo per l'intera esecuzione (default 90 secondi)

Stato di uscita del runner: 0 se la suite passa, 1 per test falliti o problemi di QEMU, 2 per errori di configurazione.

1.9.4 Ispezione e debug

Per vedere header, entry point, sezioni caricate e relocation di un programma si usano gli strumenti ARM bare-metal `readelf` (`arm-none-eabi-readelf`), `objdump`, `nm` e `size`. Il file `build/output.map` mostra i simboli del linker del kernel.

GDB non sa in quale slot è stato caricato un programma, quindi bisogna dirglielo con il comando seguente.

```
# In GDB: simboli di un programma all'indirizzo del suo slot
add-symbol-file PROGRAM.elf LOAD_ADDRESS
# Esempio: thumb_ptr caricato nello slot 1
add-symbol-file build/programs/thumb_ptr.elf 0x20003000
```

Si usa l'indirizzo dello slot in RAM, non quello del file dentro l'immagine. *Da chiarire con il professore:* il .md non dice come avviare QEMU e GDB per una sessione di debug.

1.10 Piano di lavoro consigliato

Il .md dice esplicitamente che i requisiti non sono un piano di lavoro e che ordine e divisione del lavoro li decide la squadra. Quella che segue è quindi una *proposta*: ogni passo si appoggia sui precedenti e finisce con un traguardo verificabile.

1. *Partire dalla base.* Compilare e avviare `scratch-os/project-base/` con `make run`; leggere `kernel/task.h`, il linker script della base e `build/output.map`.
   Traguardo: la base parte in QEMU, ancora senza shell né loader.

2. *Shell (R4).* Definire `task_project_configure()` e configurare la shell con `task_configure_shell()`; prompt, eco, limite di 31 caratteri, `help` ed `error: unknown command`.
   Traguardo: righe di boot, prompt e output di `help` identici a quelli di .

3. *Build dei programmi (R1).* Startup texttt, linker script all'indirizzo `0`, `-emit-relocs`, stub e `printf` propri.
   Traguardo: `arm-none-eabi-readelf` mostra un `ET_EXEC` con entry Thumb, almeno un `PT_LOAD` e le sezioni `REL`.

4. *Host tool (R2).* Comando `image` (più `flat` nella small) e ricetta `make image` aggiornata.
   Traguardo: messaggi di successo esatti; input sbagliati rifiutati con stato 2 prima di scrivere output.

5. *Lettura dell'immagine (R3) e SVC 11 e 12 (R5).*
   Traguardo: `ls` elenca i programmi; con un'immagine non valida stampa `error: no valid image`.

6. *Loader (R6, R5, R9; nella big anche B1).* SVC 13 con tutti i controlli, copia, BSS, relocation e `task_program_create`; SVC 14 nella shell. Small: file SFT1;

big: ELF letti dal kernel.
Traguardo: `run` produce la sessione di ; passano `global_state`, `thumb_ptr` e `nonzero_slot_relocation`.

7. *MPU e fine dei programmi (R7, R8; nella big anche B2).*
   Traguardo: passano `normal_return`, `explicit_exit` e `kernel_store`; nella big anche `wx_write_texttt` e `wx_exec_data`.

8. *Robustezza (R9).* Immagini e file malformati, argomenti sbagliati delle system call.
   Traguardo: passano `bad_magic`, `bad_header`, `too_large` e `bad_reloc`; nella big anche `unsupported_reloc` e `truncated_elf`.

9. *Suite completa.*
   Traguardo: la riga `SUMMARY` riporta `failed=0`, cioè 10 test su 10 (small) o 14 su 14 (big), con `build/scratch-os.axf` e `build/fs.img` ricostruiti da zero.

10. *Documentazione e demo.*
    Traguardo: la documentazione contiene tutte le voci di e la demo mostra i cinque punti richiesti.

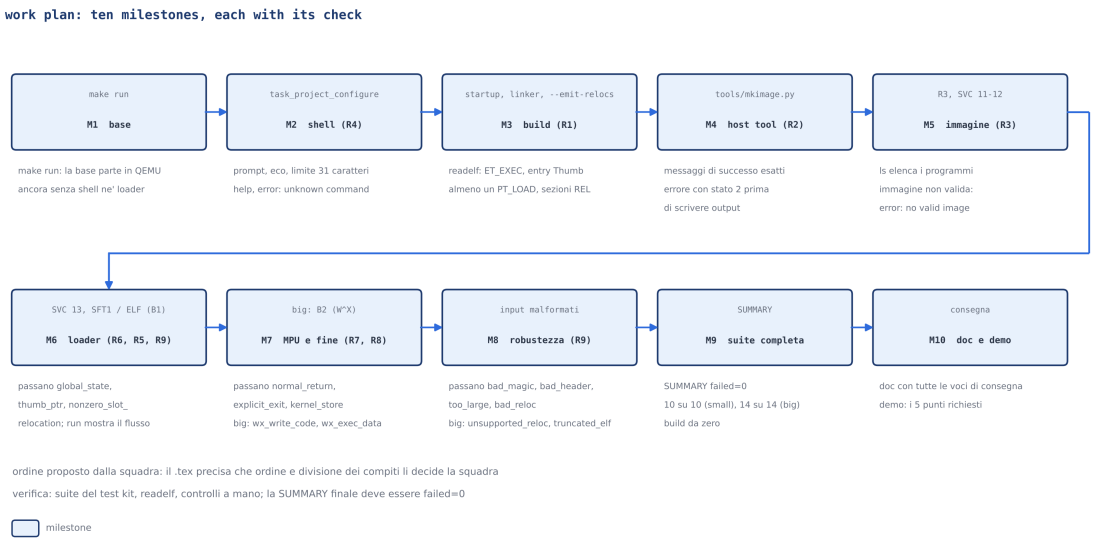


Figura 1.11: Piano di lavoro proposto: dieci milestone in sequenza, ciascuna con il criterio con cui verificare che funzioni. La squadra può cambiare l'ordine, ma ogni milestone deve chiudersi con la verifica indicata.

1.11 Test e verifica

1.11.1 Come controlla il professore

- Usa lo stesso test kit pubblicato, con la suite della versione della squadra, sullo stato del repository alla scadenza. Come kernel il runner usa `build/scratch-os.axf`.

- Ogni test avvia un QEMU nuovo e confronta tutto ciò che viene stampato sulla UART con l'output atteso. Solo i fine riga CRLF dell'host diventano LF; ogni altro byte deve coincidere.

- Le trascrizioni attese contengono anche le due righe di boot, i comandi con la loro eco, le righe del loader e di fine, e ogni prompt `scratch>` : deve coincidere l'intera trascrizione.

- I test valgono al massimo 1.50 punti, in proporzione ai test superati (). Il resto del voto viene dalla valutazione dei requisiti, dalla presentazione con demo e dalle domande individuali.

1.11.2 Il test kit

Il test kit è nella cartella `test-kit/` accanto al documento della consegna (in `Projects.md` il link è `project/test-kit/`). Contiene `run_tests.py`, i sorgenti dei programmi di test, i generatori riproducibili dei file rotti (le fixture), le definizioni delle suite (i manifest) e le trascrizioni attese.

Per ogni test il runner stampa una di queste due righe, e QEMU si ferma dopo ogni risultato:

- `PASS <name>` se il test passa;

- `FAIL <name>: <reason>` se fallisce.

L'esecuzione finisce con:

```
SUMMARY suite=<suite> passed=<passed> failed=<failed> total=<total>
```

Da chiarire con il professore: il test kit non è allegato e il .md non spiega che cosa deve contenere l'immagine passata con `-image` (quali programmi di test e con quali nomi), né come i file rotti prodotti dai generatori finiscono nell'immagine.

1.11.3 I test

La suite small ha 10 test. La suite big esegue gli stessi dieci test con file ELF e ne aggiunge quattro solo big, per un totale di 14.

I dieci test comuni:

Test	Che cosa controlla
`nonzero_slot_relocation`	Un programma padre gira nello slot 0 (base `0x20002000`, id 4); il figlio gira nello slot 1 (base `0x20003000`, id 5) e stampa `thumb_function_pointer: thumb=1 result=42`. Il padre riceve 0.
`global_state`	Slot 0, id 4: stampa `globals_bss_const: global=37 bss=0 table=16 sum=53`, poi esce.
`thumb_ptr`	Slot 0, id 4: stampa `thumb_function_pointer: thumb=1 result=42`, poi esce.
`normal_return`	Torna da `main`: `normal_return: returning from main`, poi `program normal_return exited`.
`explicit_exit`	Chiama la SVC 4: `explicit_exit: calling sys_exit`, poi `program explicit_exit exited`.
`kernel_store`	Scrive a `0x20000000`: `MemManage in loaded program: CFSR=0x82 address=0x20000000`. Poi `print_plain` riusa lo slot 0 con id 8.
`bad_magic`	Rifiutato; poi `help` mostra che la shell funziona ancora.
`bad_header`	Rifiutato; poi `help` mostra che la shell funziona ancora.
`too_large`	Un programma da 3076 byte viene rifiutato; poi `help` mostra che la shell funziona ancora.
`bad_reloc`	Una relocation fuori dal programma viene rifiutata; poi `help` mostra che la shell funziona ancora.

I quattro test solo big:

Test	Che cosa controlla
`unsupported_reloc`	Il tipo 3 stampa `loader: unsupported relocation type 3`. Poi un programma corretto gira nello slot 0 con id 4.
`truncated_elf`	Un ELF tagliato viene rifiutato senza nessuna riga di fault. Poi un programma corretto gira nello slot 0 con id 4.
`wx_write_texttt`	Una scrittura nel codice stampa `MemManage in loaded program: CFSR=0x82 address=0x20002280`. Poi un programma corretto riusa lo slot 0 con id 8.
`wx_exec_data`	Un salto nei dati stampa `MemManage in loaded program: CFSR=0x1`. Poi un programma corretto riusa lo slot 0 con id 8.

1.11.4 Come verificare da soli prima di consegnare

- Ricostruire tutto da zero (programmi, `build/fs.img`, `build/scratch-os.axf`) con i comandi scritti nella documentazione.

- Eseguire la suite della propria versione e ottenere `failed=0`; per isolare un test che fallisce, usare `-list` e `-test NAME`.

- Ripetere i test sullo stato effettivamente presente nel repository GitLab, perché la valutazione usa quello alla scadenza.
- Ispezionare i programmi con `arm-none-eabi-readelf`: header, entry point, sezioni caricate e relocation.
- Controllare i permessi MPU che risultano dopo ogni task switch.
- Verificare che sulla UART non esca nessuna riga di debug.
- Provare la demo completa: caricamento normale, relocation in uno slot diverso da 0, rifiuto di un input sbagliato, recupero da un fault e comportamento specifico della versione.

1.12 Consegna

Dove. Ogni squadra riceve account GitLab del corso: il lavoro si tiene e si consegna in quel repository.

Quando. Non c'è una data di inizio obbligatoria. Il progetto va completato entro la fine dell'anno accademico, e la valutazione usa lo stato del repository alla scadenza. Le squadre si formano entro il 16 ottobre (). *Da chiarire con il professore:* la data esatta della scadenza di consegna e quando si tiene la presentazione.

Che cosa consegnare.

- Tutti i sorgenti e i file di build che servono per ricompilare il kernel e tutti i programmi.
- `tools/mkimage.py` e `build/fs.img`.
- Il kernel eseguibile `build/scratch-os.axf`, che il test runner usa come kernel.
- Nessun file oggetto generato, tranne l'immagine richiesta e il kernel eseguibile.

Documentazione. Deve contenere:

- i prerequisiti;
- i comandi esatti di build, esecuzione e test;
- le scelte di progetto e di formato;
- come la squadra ha integrato la MPU e il codice dato del ciclo di vita;
- i limiti noti;
- la procedura esatta della demo.

Nel complesso deve dare informazioni sufficienti per riprodurre il risultato consegnato.

Presentazione e demo. È una presentazione orale di 20 minuti con tutti i membri, seguita da domande individuali e da una demo dal vivo. La demo deve mostrare: caricamento normale, relocation in uno slot diverso da 0, rifiuto di un input sbagliato, recupero da un fault e il comportamento specifico della versione.

Regole per i membri. Ogni membro deve capire tutto il progetto, anche il codice scritto dagli altri: il professore può chiedere a chiunque qualunque parte, e ogni risposta conta per il voto di quello studente.

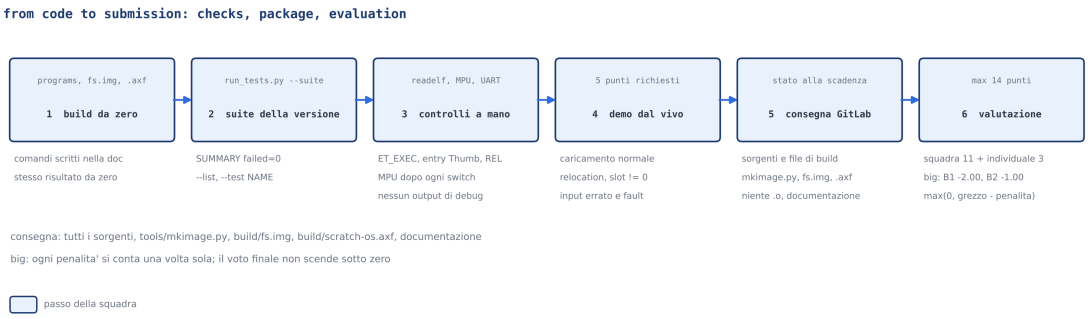


Figura 1.12: Dal codice alla consegna: build da zero, suite della propria versione con `failed=0`, controlli a mano, demo e consegna. Il voto somma squadra e individuale, con le penalità della versione big.

1.13 Valutazione

Il progetto vale da 0 a 14 punti grezzi; il massimo è lo stesso per small e big. Il professore valuta implementazione, test pubblicati, efficienza, documentazione, presentazione con demo e la comprensione dell'intero progetto da parte di ogni studente.

Requisiti implementati:

Parte	Che cosa si valuta	Max
Squadra	Immagine: IMG1 valida, ricerca nell'immagine con limiti controllati, directory controllata, programmi trovati senza copiare l'immagine (R2, R3)	1.00
Squadra	Shell e system call di immagine e avvio: comportamento esatto, errori, output e ritorno al prompt (R4, R5)	1.00
Squadra	Build dei programmi (ABI e percorso di build): startup texttt, linker script, integrazione di libreria e system call, comportamento dell'host tool della propria versione (R1, R2)	0.75
Squadra	Caricamento e relocation: copia del contenuto inizializzato, BSS a zero, controllo di entry e geometria dello stack, correzione degli indirizzi assoluti, task creato solo dopo una validazione riuscita (R6)	2.50
Squadra	Isolamento MPU e uso corretto del supporto dato per esecuzione, attesa, exit, fault, liberazione e riuso degli slot (R7, R8)	1.00
Squadra	Rifiuto di immagini e programmi malformati o non sicuri prima dell'esecuzione, senza cambiare o corrompere uno slot e senza crash del kernel (R9)	0.75
	Subtotale requisiti	7.00

Altri criteri e totale:

Parte	Che cosa si valuta	Max
Squadra	Test pubblicati della propria versione, in proporzione	1.50
Squadra	Efficienza: risorse limitate, niente copie inutili dell'immagine né allocazione dinamica, controlli ben strutturati, niente duplicazioni evitabili	0.50
Squadra	Documentazione: istruzioni di build ed esecuzione, scelte di progetto e di formato, limiti, informazioni sufficienti per riprodurre il risultato	0.75
Squadra	Presentazione e demo dal vivo: presentare il sistema completo e mostrare caricamento normale, relocation, rifiuto di input sbagliato e recupero da fault	1.25
Squadra	Lavoro di squadra (teamwork)	0.00
Subtotale di squadra (requisiti compresi), uguale per tutti i membri		11.00
Individuale	Spiegare e ragionare su qualunque parte scelta dal docente, anche scritta da altri; le domande possono riguardare progetto, formati, relocation, MPU, controlli, system call e vita di un programma	3.00
	Subtotale individuale, valutato per ogni studente	3.00
	Massimo grezzo	14.00

Regole di calcolo. Le righe valutate a giudizio usano passi di 0.25 punti. La riga dei test non viene arrotondata:

$$\text{punti test} = 1.50 \times \frac{\text{test applicabili superati}}{\text{test applicabili della suite}}$$

I test applicabili sono 10 per le squadre small e 14 per le big. Per esempio, una squadra small che supera 8 test prende $1.50 \times 8/10 = 1.20$ punti. Il lavoro di squadra (teamwork) vale zero punti, e i punti individuali non dipendono da chi dichiara di aver scritto quale parte.

Penalità della big. Per una squadra big si calcola prima il punteggio grezzo da 0 a 14, poi si sottrae una volta ogni penalità applicabile; un risultato negativo diventa zero.

Extra della big mancante o non funzionante	Penalità
B1, caricamento ELF nel kernel: impacchettamento byte per byte, gestione richiesta delle relocation, messaggio con il tipo numerico, controlli di intervallo su ogni tabella, record e intervallo usati	−2.00
B2, protezione W^X di codice e dati, in entrambe le direzioni	−1.00

$$\text{voto finale} = \max(0,\ \text{punteggio grezzo} - \text{penalità big applicabili})$$

Il caricamento ELF è un solo extra e può dare una sola penalità. Anche W^X è un solo extra: se manca una delle due direzioni non si prende una seconda penalità.

Che cosa fa perdere punti.

- Ogni test fallito; basta un solo byte diverso nella trascrizione per far fallire un test.
- Copie inutili dell'immagine, allocazione dinamica, controlli confusi o duplicati (efficienza).
- Una documentazione incompleta o che non permette di riprodurre il risultato.
- Una demo che non mostra tutti i punti richiesti.
- Risposte individuali deboli, anche su parti scritte da altri.
- Nella big, un extra mancante o non funzionante (B1: −2.00; B2: −1.00).

1.14 Errori comuni e consigli del professore

- *L'immagine della base non è valida.* È un segnaposto volutamente non IMG1, e `make run` usa un'immagine già esistente: la build deve prima creare una vera IMG1.
- *La ricetta `make image` va cambiata.* È solo un punto di partenza: deve usare il comando `image` esatto, con argomenti `NAME=PROGRAM.elf` espliciti.
- *GDB e programmi caricati.* GDB non sa dove è stato caricato un programma: con `add-symbol-file` gli si dà l'indirizzo dello slot in RAM, non quello del file nell'immagine.

- *Regioni MPU sul Cortex-M3.* La dimensione di una regione è una potenza di 2 e la sua base deve essere allineata a quella dimensione. Quando due regioni si sovrappongono vince quella con il numero più alto. Bisogna controllare i permessi risultanti dopo ogni task switch.
- *W^X nella big.* Il codice deve rifiutare le scritture, dati e stack devono rifiutare il fetch di istruzioni; una regione sovrapposta con lettura, scrittura ed esecuzione rompe W^X. L'argomento `executable` da solo non basta.
- *Bit 0 Thumb.* Quando si corregge un puntatore a funzione il bit 0 deve restare com'è.
- *La parola da correggere può puntare fuori dal programma.* Non è un errore; conta invece che `slot_base + original_word` non vada in overflow.
- *Overflow nei calcoli.* Calcoli come `directory_offset + count * 24` e `texttt_size + data_size + bss_size` vanno protetti, e così ogni calcolo di intervallo, anche nella SVC 1.
- *Relocation da ignorare.* Quelle delle sezioni non allocated, come il debug (compreso `R_ARM_NONE`), si ignorano; nella big i loro record devono comunque stare dentro il file.
- *Il programma può partire subito.* Dopo la SVC 13 il nuovo programma può girare prima che il creatore chiami la SVC 14: lo stato di fine resta salvato nello slot.
- *L'attesa non riserva lo slot*, e più task possono aspettare lo stesso programma.
- *Righe stampate dal codice dato.* La riga `loader:` la stampa già `task_program_create`, e i messaggi di fine e di fault vengono dal codice dato: stamparli di nuovo rompe la trascrizione.
- *Niente output di debug* sulla UART: cambierebbe le trascrizioni.
- *Il codice dato resta al comando.* Si possono aggiungere moduli e campi, ma i fault durante il salvataggio dello stack frame, i fault nel kernel e quelli fuori dai programmi caricati devono restare non recuperabili.
- *Tutti devono sapere tutto.* Le domande individuali possono riguardare progetto, formati, relocation, MPU, controlli, system call e vita di un programma, anche nelle parti scritte da altri.

1.15 Checklist finale prima della consegna

☐ Squadra registrata nel Google Spreadsheet entro il 16 ottobre.

☐ R1: ogni programma di `programs/` compilato a parte, linkato a `0` con `-emit-relocs`, con startup texttt, stub, `printf` e helper propri; `_start` chiama `main`; il ritorno da `main` equivale a `sys_exit()`.

☐ R1: la build ricrea programmi e `build/fs.img` dai sorgenti, con GCC ARM bare-metal, sempre allo stesso modo.

- ☐ R2: `tools/mkimage.py` crea `build/fs.img` in formato IMG1 con i comandi e i messaggi esatti, e rifiuta l'input sbagliato tramite `argparse` (stato 2); nella small c'è anche `flat`.
- ☐ R3: l'immagine viene letta a `0x00200000` e controllata, mai scritta né copiata.
- ☐ R4: la shell accetta solo `help`, `ls` e `run <name>`, stampa prompt e messaggi esatti e non stampa il prompt mentre un programma gira.
- ☐ R5: le SVC da 11 a 14 restituiscono esattamente i risultati previsti.
- ☐ R6: primo slot libero, copia di codice e dati, BSS a zero, relocation, controlli di entry e stack, e solo dopo `task_program_create`.
- ☐ R7: ogni programma gira unprivileged e la MPU gli lascia solo il proprio slot in RAM.
- ☐ R8: ritorno da `main`, `sys_exit()` e fault recuperabili liberano subito lo slot e riportano il prompt, usando il codice dato.
- ☐ R9: input malformati rifiutati prima di copiare o eseguire, senza crash e senza cambiare slot.
- ☐ Solo big, B1: immagine con gli ELF copiati byte per byte, ELF controllati e caricati dal kernel, messaggio `loader: unsupported relocation type <type>`.
- ☐ Solo big, B2: W^X in entrambe le direzioni.
- ☐ Suite della propria versione superata: 10 test su 10 (small) o 14 su 14 (big).
- ☐ Nel repository GitLab del corso ci sono tutti i sorgenti e i file di build per ricompilare kernel e programmi.
- ☐ Nel repository ci sono `tools/mkimage.py`, `build/fs.img` e `build/scratch-os.axf`.
- ☐ Nessun file oggetto generato, oltre all'immagine e al kernel eseguibile.
- ☐ Documentazione con prerequisiti, comandi esatti di build, esecuzione e test, scelte di progetto e di formato, integrazione di MPU e codice dato, limiti noti e procedura esatta della demo.
- ☐ Codice dato del ciclo di vita e dei fault non sostituito né copiato.
- ☐ Nessun output di debug e nessun comando extra; programmi e test privati non cambiano comandi, output, formati e test.
- ☐ Progetto completato entro la fine dell'anno accademico (conta lo stato del repository alla scadenza).
- ☐ Presentazione di 20 minuti preparata con tutti i membri; demo con caricamento normale, relocation in uno slot diverso da 0, rifiuto di input sbagliato, recupero da fault e comportamento della versione.
- ☐ Ogni membro sa spiegare ogni parte del progetto.

1.16 Mappa dei file di project-base — scratch-OS (Cortex-M3, QEMU mps2-an385)

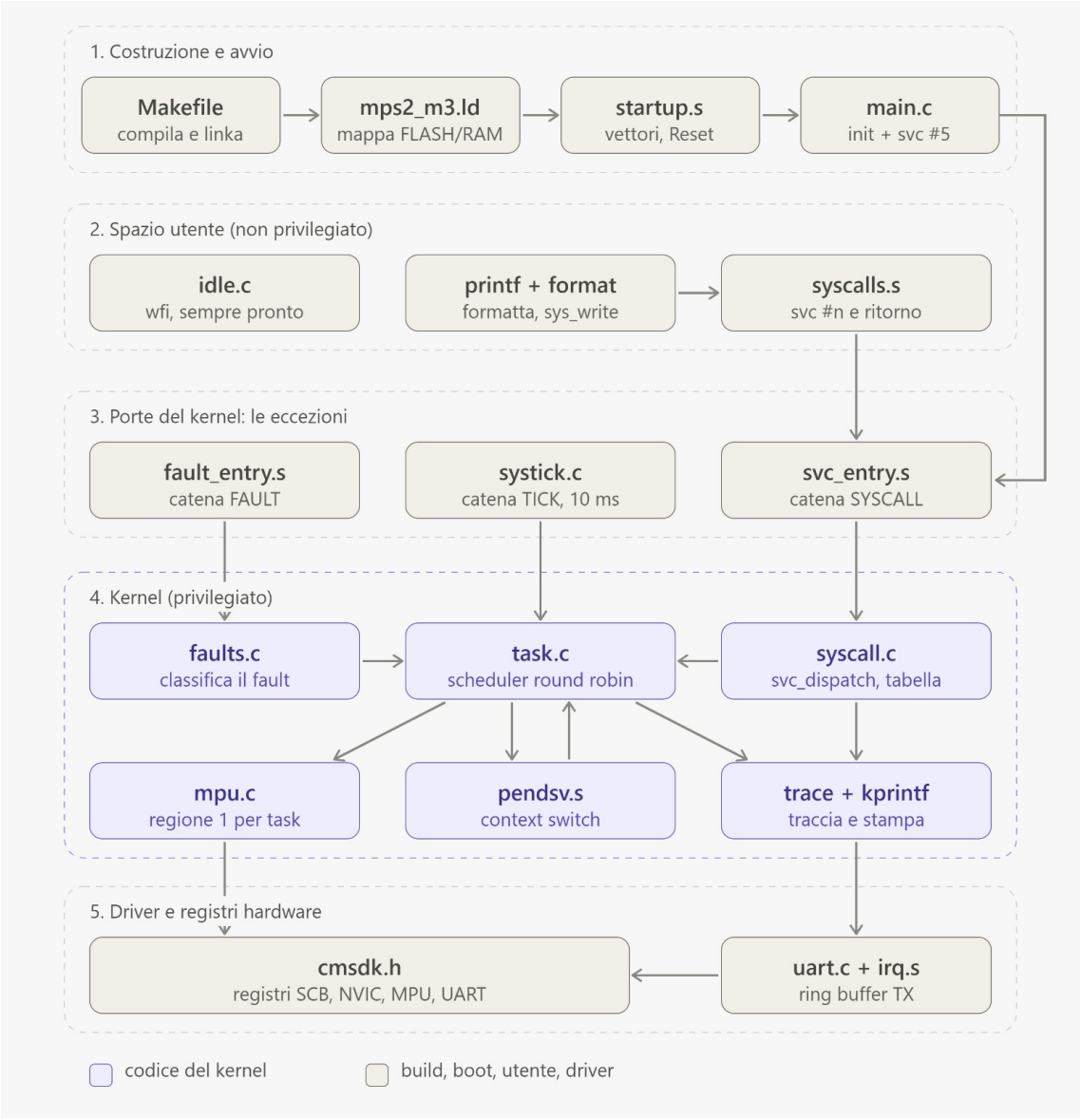


Figura 1.13: Giulio vaffanculo

1.16.1 Build e linker

`Makefile` — Compila ogni `.c`/`.s` trovato in `boot kernel drivers lib src`, linka `build/scratch-os.axf` e lancia QEMU mps2-an385.

- **Contiene:** variabili `CC`, `QUANTUM` (default 5), `BUILD_DIR`, `BIN`, `IMAGE`, `IMAGE_ADDR`, `IMAGE_TOOL`, `PROGRAM_DIR`, `SRC_DIRS`, `SOURCES`, `OBJECTS`, `CFLAGS`, `LDFLAGS`, `QEMU`; target `all`, `image`, `run`, `debug`, `run-trace`, `test`, `clean`

- **Dipende da:** `scripts/mps2_m3.ld`; `tools/mkimage.py` (non presente nella base) e `programs/` per il target `image`; `arm-none-eabi-gcc`, `qemu-system-arm`, `python3`; passa `-DQUANTUM` a `kernel/task.h`; usa `-Iinclude` e `-I.`
- **Usato da:** nessun file (è il punto d'ingresso della build)
- **Tocca:** scrive solo in `build/` (`scratch-os.axf`, `output.map`, `fs.img` segnaposto); `clean` fa `rm -rf build`; QEMU carica `fs.img` a `0x00200000`

`scripts/mps2_m3.ld` — Linker script: FLASH a `0x0` (4M) e RAM a `0x20000000` (4M); riserva le partizioni di shell e idle e 4 slot da 4 KB per i programmi.

- **Contiene:** `MEMORY`, `ENTRY(Reset_Handler)`, sezioni `.isr_vector .text .shell_stack .shell_data .shell_pad .idle_stack .idle_data .idle_pad .program_slots .data .bss`; simboli `_estack`, `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss`, `_shell_*`, `_idle_*`, `__program_slot_*`, `__fs_image_start`, `__StackLimit`; 4 `ASSERT`
- **Dipende da:** `Reset_Handler` e sezione `.isr_vector` (`boot/startup.s`); `idle.o` (`src/idle.c`); `shell.o` (non presente nella base)
- **Usato da:** `Makefile` (`-T`); `boot/startup.s` usa `_estack`, `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss`; `kernel/task.c` usa `_idle_*` e `__program_slots_start`; `_shell_*`, `__program_slot_N_*` e `__fs_image_start` non sono usati da nessun file visto
- **Tocca:** solo il layout: stack kernel da 4 KB in cima alla RAM, partizioni da 4 KB allineate (stack da 1 KB in basso), `.data` caricata in FLASH ed eseguita in RAM

1.16.2 Boot e definizioni hardware

`boot/startup.s` — Codice di boot in assembly: tabella dei vettori, `Reset_Handler` (costruisce il runtime C e configura la CPU) e `Default_Handler`. Alla fine chiama `main()`.

- **Contiene:** `isr_vector` (48 word, sezione `.isr_vector`), `Reset_Handler` (etichette `copy_data`, `zero_bss`, `hang`), `Default_Handler`; alias `.weak`/`.thumb_set` di 9 handler di sistema (`NMI`, `HardFault`, `MemManage`, `BusFault`, `UsageFault`, `SVC`, `DebugMon`, `PendSV`, `SysTick`) e di 10 handler di dispositivo (`UART0RX/TX`, `UART1RX/TX`, `UART2RX/TX`, `TIMER0`, `TIMER1`, `DUALTIMER`, `UARTOVF`)
- **Dipende da:** `_estack`, `_sidata`, `_sdata`, `_edata`, `_sbss`, `_ebss` (`scripts/mps2_m3.ld`); `main` (`src/main.c`); non include `cmsdk.h` (usa indirizzi scritti a mano)
- **Usato da:** `scripts/mps2_m3.ld` (`ENTRY` e `.isr_vector` a `0x0`); alias sostituiti da `SVC_Handler` (`kernel/svc_entry.s`), `PendSV_Handler` (`kernel/pendsv.s`), `SysTick_Handler` (`drivers/systick.c`), `UART0TX_Handler` (`drivers/uart.c`), `MemManage`, `BusFault`, `UsageFault` e `HardFault` (`kernel/fault_entry.s`); restano deboli `NMI`, `DebugMon`, `UART0RX`, `UART1RX/TX`, `UART2RX/TX`, `TIMER0`, `TIMER1`, `DUALTIMER`, `UARTOVF`

- **Tocca:** PRIMASK (`cpsid`/`cpsie`), SP (= `_estack`); copia `.data` da FLASH a RAM e azzera `.bss`; VTOR (`0xE000ED08`), SHCSR (`0xE000ED24`: abilita MemManage, BusFault, UsageFault), CCR (`0xE000ED14`: DIV_0_TRP), priorità in `0xE000ED18..23` (fault `0x00`, SVC `0x80`, PendSV e SysTick `0xE0`); usa r0–r3

`include/cmsdk.h` — Definizioni dei registri di mps2-an385: struct sovrapposte ai periferici (SysTick, NVIC, SCB, MPU, UART0–2, TIMER0–1, WDOG, GPIO0, FPGAIO), con macro dei bit, costanti `*_IRQn` e `_Static_assert` sugli offset; molte macro non sono usate da nessun file visto.

- **Dipende da:** `stdint.h`, `stddef.h`. **Usato da:** `drivers/systick.c`, `drivers/uart.c`, `kernel/mpu.c`, `kernel/faults.c`, `kernel/task.c`. **Tocca:** niente (solo definizioni)

1.16.3 Applicazione di base (`src`)

`src/main.c` — Entry point del kernel: inizializza UART, SysTick e MPU, poi lancia i task con `svc #5` (`SYS_START_TASKS`). Non ritorna mai.

- **Contiene:** `main()`
- **Dipende da:** `drivers/uart.h` (`uart_init`), `drivers/systick.h` (`systick_init`), `kernel/kprintf.h` (`kprintf`), `kernel/mpu.h` (`mpu_init`); non include `lib/syscalls.h`: il numero 5 è scritto a mano e va tenuto coerente con `SYS_START_TASKS`; l'`svc` è servita da `kernel/svc_entry.s` e `kernel/syscall.c`
- **Usato da:** `Reset_Handler` in `boot/startup.s` (`bl main`)
- **Tocca:** indirettamente UART, SysTick (tick da 10 ms) e MPU tramite le init; `svc #5` genera l'eccezione SVC con clobber `memory`

`src/tasks.h` — Dichiara l'idle task, l'unico avviato dalla base.

- **Contiene:** guard `TASKS_H`, prototipo `idle_task()`
- **Dipende da:** niente
- **Usato da:** `src/idle.c` (include); `kernel/task.c` (include, usa `idle_task` come `.entry` dell'idle TCB)
- **Tocca:** niente (solo dichiarazione)

`src/idle.c` — Idle task: loop infinito con `wfi`, gira quando nessun task è READY; non privilegiato, con stack e partizione MPU propri.

- **Contiene:** `idle_task()`
- **Dipende da:** `src/tasks.h`; istruzione `wfi` (asm inline); `scripts/mps2_m3.ld` lo mette nelle sezioni `.idle_*` tramite il nome `idle.o`
- **Usato da:** `kernel/task.c` (`tasks[IDLE_TASK_INDEX].entry`)
- **Tocca:** nessuna variabile; `wfi` ferma la CPU fino al prossimo interrupt (SysTick); usa lo stack da 1 KB della partizione idle

1.16.4 Driver

drivers/systick.h — Interfaccia del driver SysTick: il timer del core usato come tick di sistema da 10 ms.

- **Contiene:** guard `SYSTICK_H`, macro `TICK_MS` (10), prototipi `systick_init()`, `SysTick_Handler()`, `systick_uptime_ms()`
- **Dipende da:** `stdint.h`
- **Usato da:** `src/main.c`, `drivers/systick.c`, `kernel/syscall.c` (include)
- **Tocca:** niente (solo dichiarazioni)

drivers/systick.c — Configura SysTick a 25 MHz per un'eccezione ogni 10 ms. L'handler aggiorna l'uptime e avvisa lo scheduler.

- **Contiene:** `CPU_CLOCK_HZ`, `uptime_ms` (static volatile), `systick_init()`, `SysTick_Handler()`, `systick_uptime_ms()`
- **Dipende da:** `drivers/systick.h`; `cmsdk.h` (`SysTick->LOAD/VAL/CTRL`, `SysTick_CTRL_*`); `kernel/task.h` (`task_increment_tick`, definita in `kernel/task.c`)
- **Usato da:** `src/main.c` (`systick_init`); `kernel/syscall.c` (`systick_uptime_ms`); `SysTick_Handler` sostituisce l'alias debole di `boot/startup.s` (vettore 15)
- **Tocca:** SysTick `LOAD` (249999), `VAL` (0), `CTRL` (ENABLE, TICKINT, CLKSOURCE); variabile `uptime_ms`

drivers/uart.h — Interfaccia del driver UART0: invio polled o con ring buffer e interrupt, ricezione solo polled.

- **Contiene:** guard `UART_H`, prototipi `uart_init`, `uart_putc_polled`, `uart_getc_polled`, `uart_write`, `UART0TX_Handler`
- **Dipende da:** niente
- **Usato da:** `src/main.c`, `drivers/uart.c`, `kernel/kprintf.c`, `kernel/syscall.c` (include)
- **Tocca:** niente (solo dichiarazioni)

drivers/uart.c — Driver UART0: invio con ring buffer da 64 byte pilotato dall'interrupt TX, più invio e ricezione polled.

- **Contiene:** `TX_BUF_SIZE`, `tx_buf`, `tx_head`, `tx_tail`, `tx_busy`, `tx_empty`, `tx_full`, `tx_pop`, `uart_init`, `uart_putc_polled`, `uart_getc_polled`, `uart_write`, `UART0TX_Handler`
- **Dipende da:** `drivers/uart.h`; `cmsdk.h` (`CMSDK_UART0`, `UART_CTRL_*`, `UART_STATE_*`, `UART_INT_TX`, `NVIC`, `UART0TX_IRQn`); `kernel/irq.h` (`irq_save`, `irq_restore`, implementate in `kernel/irq.s`)

- **Usato da:** `src/main.c` (`uart_init`); `kernel/kprintf.c` (`uart_putc_polled`); `kernel/syscall.c` (`uart_write`, `uart_getc_polled`); `UART0TX_Handler` sostituisce l'alias debole di `boot/startup.s` (vettore 17)

- **Tocca:** UART0 `BAUDDIV` (16), `CTRL` (TXEN, RXEN, TXIRQEN), `DATA`, `INTSTATUS`; NVIC `IPR` (0xC0) e `ISER` per UART0TX; PRIMASK (via `irq_save`); globali `tx_buf`, `tx_head`, `tx_tail`, `tx_busy`

1.16.5 Kernel

`kernel/task.h` — Interfaccia dello scheduler: TCB, tabella dei task, stati e funzioni del kernel (round robin preemptive con time slice `QUANTUM`).

- **Contiene:** guard `TASK_H`, `QUANTUM` (default 5, ridefinito da `-DQUANTUM`), `enum task_state`, `struct tcb` (`sp` deve restare il primo campo), `PROGRAM_SLOT_COUNT`, `SHELL_TASK_INDEX`, `PROGRAM_TASK_FIRST`, `IDLE_TASK_INDEX`, `NUM_TASKS`; extern `tasks`, `tcb_current`, `tick_count`; prototipi `task_start_first`, `task_increment_tick`, `task_request_switch`, `task_block_until`, `task_finish_current`, `task_project_configure`, `task_configure_shell`, `task_program_create`, `task_program_slot_free`, `task_wait_program`, `task_fault_current`, `task_current_is_program`, `task_switch_context`

- **Dipende da:** `stdint.h`; definizioni in `kernel/task.c`; `QUANTUM` dal `Makefile`

- **Usato da:** `drivers/systick.c`, `kernel/syscall.c`, `kernel/task.c`, `kernel/faults.c`, `kernel/trace.c` (include); `kernel/pendsv.s` (non lo include, ma assume `sp` a offset 0 e usa `tcb_current`)

- **Tocca:** niente (solo dichiarazioni)

`kernel/task.c` — Scheduler round robin: tabella dei task, frame iniziali, tick, scelta del prossimo task, slot dei programmi caricati (id con generazione, attesa, fault). Gira privilegiato dentro SVCall, SysTick e PendSV.

- **Contiene:** `tasks[]`, `tcb_current`, `tick_count`, `quantum_left`, `rr_last`, `program_names`, `generations`, `completed_id`, `completed_status`; `partition_init`, `task_init_stack`, `task_project_configure` (weak), `task_configure_shell`, `task_start_first`, `task_request_switch`, `task_increment_tick`, `task_block_until`, `program_index`, `task_current_is_program`, `task_program_slot_free`, `task_program_create`, `task_wait_program`, `complete_program`, `task_finish_current`, `task_fault_current`, `task_switch_context`

- **Dipende da:** `cmsdk.h` (`SCB->ICSR`, `SCB_ICSR_PENDSVSET`); `kernel/task.h`; `kernel/mpu.h` (`mpu_set_task_region`); `kernel/kprintf.h`; `kernel/trace.h` (`trace_record_switch`); `src/tasks.h` (`idle_task`); `lib/syscalls.h` (`task_exit`, in `lib/syscalls.s`); simboli del linker `_idle_part_start`, `_idle_stack_top`, `_idle_part_end`, `_idle_data_*`, `__program_slots_start`; definizione forte di `task_project_configure` (shell degli studenti, non presente nella base)

- **Usato da:** `drivers/systick.c` (`task_increment_tick`); `kernel/syscall.c` (`task_start_first`, `task_request_switch`, `task_block_until`, `task_finish_current`, `task_wait_program`, `tcb_current`, `tick_count`); `kernel/pendsv.s` (`tcb_current`, `task_switch_context`); `kernel/faults.c` (`task_current_is_program`, `task_fault_current`); `kernel/trace.c` (`tasks`, `tick_count`)
- **Tocca:** i globali elencati; `ICSR.PENDSVSET`; MPU regione 1 a ogni switch; PSP e `CONTROL.nPRIV` (una volta, in `task_start_first`); copia i dati iniziali di shell e idle; scrive il frame iniziale (xPSR, pc, lr = `task_exit`, r0–r12) sullo stack di ogni task; stampa con `kprintf`; scrive nel r0 salvato di un task in attesa (`wait_result`)

`kernel/syscall.h` — Lato kernel delle system call: dichiara `svc_dispatch()`.

- **Contiene:** guard `SYSCALL_H`, prototipo `svc_dispatch()`
- **Dipende da:** `stdint.h`
- **Usato da:** `kernel/syscall.c` (include); `kernel/svc_entry.s` (`bl svc_dispatch`)
- **Tocca:** niente (solo dichiarazione)

`kernel/syscall.c` — Smista `svc #n` leggendo `n` dal byte prima del pc salvato, esegue il servizio dalla tabella e scrive il risultato nel r0 salvato. `SYS_START_TASKS` è gestito a parte. `TABLE_SIZE` vale 15 (il commento dice 11); i servizi 11–13 non hanno voce e rispondono `-1`.

- **Contiene:** `syscall_fn`, `in_range`, `sys_write_impl`, `sys_read_impl`, `sys_uptime_impl`, `sys_yield_impl`, `sys_delay_impl`, `sys_delay_until_impl`, `sys_exit_impl`, `sys_tick_count_impl`, `sys_trace_dump_impl`, `sys_program_wait_impl`, `table`, `TABLE_SIZE`, `EXC_RETURN_PSP_BIT`, `svc_dispatch`
- **Dipende da:** `kernel/syscall.h`; `lib/syscalls.h` (`SYS_*`); `drivers/uart.h` (`uart_write`, `uart_getc_polled`); `drivers/systick.h` (`systick_uptime_ms`); `kernel/task.h` (`tcb_current`, `tick_count`, `task_request_switch`, `task_block_until`, `task_finish_current`, `task_start_first`, `task_wait_program`, campi `part_start`/`part_end`); `kernel/trace.h` (`trace_print`)
- **Usato da:** `kernel/svc_entry.s` (chiama `svc_dispatch`)
- **Tocca:** `frame[0]` (r0 salvato del chiamante); legge `frame[6]` e il byte a `pc-2`; legge PSP (`sys_program_wait_impl`); scrive `*last_wake` nella partizione del task; stato dei task solo tramite le funzioni di `kernel/task.c`

`kernel/svc_entry.s` — `SVC_Handler`: sceglie MSP o PSP dal bit 2 di EXC_RETURN, chiama `svc_dispatch(frame, exc_return)` e termina l'eccezione con il valore restituito.

- **Contiene:** `SVC_Handler`
- **Dipende da:** `svc_dispatch` (`kernel/syscall.c`)

- **Usato da:** vettore 11 di `boot/startup.s` (sostituisce l'alias debole); entrato da ogni `svc` di `lib/syscalls.s` e di `src/main.c`

- **Tocca:** legge `lr`, MSP e PSP; usa r0 e r1; non scrive sullo stack; `bx r0` con EXC_RETURN

`kernel/pendsv.s` — `PendSV_Handler`: il context switch. Salva r4–r11 sullo stack del task, chiama `task_switch_context()`, ripristina r4–r11 del nuovo task e rientra con `0xFFFFFFFD`.

- **Contiene:** `PendSV_Handler`, `.ltorg`

- **Dipende da:** `tcb_current` e `task_switch_context` (`kernel/task.c`, dichiarati in `kernel/task.h`); campo `sp` a offset 0 di `struct tcb`

- **Usato da:** vettore 14 di `boot/startup.s` (sostituisce l'alias debole); reso pending da `task_request_switch()` (`kernel/task.c`), chiamata da SysTick e da `kernel/syscall.c`

- **Tocca:** PSP; r0–r2 e r4–r11; 8 word sullo stack del task; `tcb_current->sp`; priorità `0xE0` impostata in `boot/startup.s`

`kernel/mpu.h` — Interfaccia dell'MPU: regione 0 = FLASH in sola lettura, regione 1 = partizione del task corrente.

- **Contiene:** guard `MPU_H`, prototipi `mpu_init()`, `mpu_set_task_region()`

- **Dipende da:** `stdint.h`

- **Usato da:** `src/main.c` (`mpu_init`), `kernel/mpu.c`, `kernel/task.c` (`mpu_set_task_region`, da `task_start_first` e `task_switch_context`)

- **Tocca:** niente (solo dichiarazioni)

`kernel/mpu.c` — Configura la regione 0 (FLASH, 4 MB, sola lettura) e abilita l'MPU con PRIVDEFENA; `mpu_set_task_region` punta la regione 1 alla partizione del task (RW, XN salvo `executable`).

- **Contiene:** `mpu_init`, `mpu_set_task_region`

- **Dipende da:** `stdint.h`; `kernel/mpu.h`; `cmsdk.h` (`MPU->RBAR/RASR/CTRL`, `MPU_RBAR_VALID`, `MPU_RASR_*`, `MPU_AP_*`, `MPU_CTRL_*`); `__builtin_ctz`

- **Usato da:** `src/main.c` (`mpu_init`); `kernel/task.c` (`mpu_set_task_region`)

- **Tocca:** MPU `RBAR` e `RASR` (regioni 0 e 1), `CTRL` (= 5); istruzioni `dsb` e `isb`

`kernel/fault_entry.s` — Entry point dei fault: una macro `FAULT` genera quattro handler che passano a C il tipo di fault (r0) e EXC_RETURN (r1).

- **Contiene:** macro `FAULT`; `MemManage_Handler` (1), `BusFault_Handler` (2), `UsageFault_Handler` (3), `HardFault_Handler` (4)
- **Dipende da:** `fault_dispatch` (`kernel/faults.c`)
- **Usato da:** vettori 3–6 di `boot/startup.s` (sostituiscono gli alias deboli); i fault configurabili sono abilitati da SHCSR in `Reset_Handler`
- **Tocca:** r0 e r1; salta con `b` (non `bl`), quindi `lr` resta EXC_RETURN e il ritorno di `fault_dispatch` termina l'eccezione; nessuno stack

`kernel/faults.c` — Classifica il fault leggendo CFSR. Se viene da un programma caricato e non è un errore di stacking, stampa, cancella CFSR e termina il task; altrimenti stampa e si ferma.

- **Contiene:** `EXC_RETURN_PSP_BIT`, `BUS_STACK_ERRORS`, `halt` (static), `fault_dispatch`
- **Dipende da:** `cmsdk.h` (`SCB->CFSR/HFSR/MMFAR`, `SCB_CFSR_MSTKERR`, `SCB_CFSR_MUNSTKERR`, `SCB_CFSR_MMARVALID`); `kernel/kprintf.h`; `kernel/task.h` (`task_current_is_program`, `task_fault_current`, in `kernel/task.c`)
- **Usato da:** `kernel/fault_entry.s` (salto a `fault_dispatch`)
- **Tocca:** SCB `CFSR` (riscritto per azzerare i flag); legge `HFSR` e `MMFAR`; UART via `kprintf`; stato del task corrente via `task_fault_current`; nel caso irrecuperabile resta in un ciclo infinito

`kernel/irq.h` — Interfaccia per mascherare e riabilitare gli interrupt attorno a una sezione critica.

- **Contiene:** guard `IRQ_H`, prototipi `irq_save()`, `irq_restore()`
- **Dipende da:** `stdint.h`; implementazione in `kernel/irq.s`
- **Usato da:** `drivers/uart.c` (include)
- **Tocca:** niente (solo dichiarazioni)

`kernel/irq.s` — `irq_save` legge PRIMASK e maschera gli interrupt; `irq_restore` rimette il valore salvato.

- **Contiene:** `irq_save`, `irq_restore`
- **Dipende da:** niente (solo istruzioni `mrs`, `msr`, `cpsid`)
- **Usato da:** `drivers/uart.c` (`uart_write`), tramite `kernel/irq.h`
- **Tocca:** PRIMASK; r0 (valore di ritorno e argomento); nessuno stack

kernel/kprintf.h — Interfaccia della print del kernel, che usa la UART in polling e funziona anche dentro un fault handler.

- **Contiene:** guard KPRINTF_H, prototipo kprintf()
- **Dipende da:** niente
- **Usato da:** src/main.c, kernel/kprintf.c, kernel/faults.c, kernel/trace.c, kernel/task.c
- **Tocca:** niente (solo dichiarazione)

kernel/kprintf.c — kprintf formatta con vformat e invia ogni carattere con uart_putc_polled.

- **Contiene:** kprintf_out (static), kprintf
- **Dipende da:** kernel/kprintf.h; lib/format.h (vformat, e stdarg.h); drivers/uart.h (uart_putc_polled)
- **Usato da:** src/main.c, kernel/faults.c, kernel/trace.c, kernel/task.c
- **Tocca:** nessun globale; UART0 DATA (attende TXBF) tramite uart_putc_polled

kernel/trace.h — Interfaccia della traccia dello scheduler: registra ogni cambio di task e lo stampa con un diagramma di Gantt.

- **Contiene:** guard TRACE_H, prototipi trace_record_switch(), trace_print()
- **Dipende da:** stdint.h
- **Usato da:** kernel/trace.c, kernel/syscall.c (trace_print), kernel/task.c (trace_record_switch, da task_switch_context)
- **Tocca:** niente (solo dichiarazioni)

kernel/trace.c — Buffer ad anello da 64 record (tick, task di partenza, task di arrivo). trace_print stampa il formato TRACE BEGIN/END e poi il Gantt.

- **Contiene:** struct trace_record, TRACE_SIZE, buffer, count, trace_record_switch, stored, oldest_plus, owner_at, GANTT_COLUMNS, draw_gantt, trace_print
- **Dipende da:** kernel/trace.h; kernel/task.h (tick_count, tasks, NUM_TASKS, definiti in kernel/task.c); kernel/kprintf.h
- **Usato da:** kernel/syscall.c (trace_print, per SYS_TRACE_DUMP); kernel/task.c (trace_record_switch)
- **Tocca:** globali buffer e count (nessuna protezione: PendSV e SVCall non si interrompono); legge tick_count e i nomi in tasks[]; UART via kprintf

1.16.6 Libreria lato utente (`lib`)

`lib/format.h` — Interfaccia della formattazione testo: numeri e stringa di formato diventano caratteri passati a una funzione `out` del chiamante.

- **Contiene:** guard `FORMAT_H`, prototipi `utoa_dec`, `utoa_hex`, `vformat`
- **Dipende da:** `stdarg.h`, `stdint.h`
- **Usato da:** `lib/format.c`, `lib/printf.c`, `kernel/kprintf.c`
- **Tocca:** niente (solo dichiarazioni)

`lib/format.c` — Implementa `utoa_dec`, `utoa_hex` e `vformat` (conversioni `c s d u x` e il segno di percentuale, senza larghezza né precisione).

- **Contiene:** `utoa_base` (static), `utoa_dec`, `utoa_hex`, `put_string` (static), `vformat`
- **Dipende da:** `lib/format.h`; la funzione `out` e il `ctx` ricevuti dal chiamante
- **Usato da:** `lib/printf.c` e `kernel/kprintf.c` (`vformat`)
- **Tocca:** nessuno stato globale né registri hardware; solo i buffer del chiamante e il proprio stack (`tmp[10]`, `num[12]`)

`lib/printf.h` — Interfaccia dell'output e delle attese lato utente, costruite solo sulle system call.

- **Contiene:** guard `PRINTF_H`, prototipi `write_all`, `printf`, `sleep_ms`, `spin_ticks`
- **Dipende da:** `stdint.h`
- **Usato da:** `lib/printf.c`; shell e programmi (non presenti nella base)
- **Tocca:** niente (solo dichiarazioni)

`lib/printf.c` — Output non privilegiato: `printf` formatta con `vformat` in un buffer da 64 byte e lo invia con `sys_write`. Include `sleep_ms` e `spin_ticks`.

- **Contiene:** `PRINTF_BUF_SIZE`, `write_all`, `struct printf_ctx`, `printf_out` (static), `printf`, `sleep_ms`, `spin_ticks`
- **Dipende da:** `stdarg.h`; `lib/printf.h`; `lib/format.h` (`vformat`); `lib/syscalls.h` (`sys_write`, `sys_uptime`, `task_tick_count`, in `lib/syscalls.s`); istruzione `wfi`
- **Usato da:** nessun file della base; pensato per shell e programmi (non presenti)
- **Tocca:** nessun globale; `struct printf_ctx` (circa 72 byte) sullo stack del chiamante; `wfi`; indirettamente SVC tramite le system call

lib/syscalls.h — Header condiviso tra kernel e applicazioni: numeri dei servizi e prototipi delle system call.

- **Contiene:** guard `SYSCALLS_H`, macro `SYS_WRITE` (1) ... `SYS_PROGRAM_WAIT` (14), con `SYS_START_TASKS` = 5; prototipi `sys_write`, `sys_read`, `sys_uptime`, `sys_exit`, `task_yield`, `task_delay`, `task_delay_until`, `task_tick_count`, `trace_dump`, `sys_image_count`, `sys_image_name`, `sys_program_run`, `sys_program_wait`, `task_exit`
- **Dipende da:** `stdint.h`
- **Usato da:** `lib/printf.c`, `kernel/syscall.c`, `kernel/task.c` (include); `lib/syscalls.s` e `src/main.c` ne ripetono a mano i numeri
- **Tocca:** niente (solo dichiarazioni)

lib/syscalls.s — Lato utente delle system call: ogni funzione fa `svc #n` e poi `bx lr`; argomenti in r0–r2, risultato in r0. Non c'è stub per `svc #5` (lo scrive `main.c`).

- **Contiene:** `sys_write`, `sys_read`, `sys_uptime`, `sys_exit`, `task_yield`, `task_delay`, `task_delay_until`, `trace_dump`, `task_tick_count`, `sys_image_count`, `sys_image_name`, `sys_program_run`, `sys_program_wait`, `task_exit`
- **Dipende da:** numeri `svc` scritti a mano e coerenti con `lib/syscalls.h`; `SVC_Handler` (`kernel/svc_entry.s`) e `svc_dispatch` (`kernel/syscall.c`); `task_exit` chiama `sys_exit`
- **Usato da:** `lib/printf.c` (`sys_write`, `sys_uptime`, `task_tick_count`); `kernel/task.c` usa l'indirizzo di `task_exit` come lr iniziale di ogni task
- **Tocca:** nessun registro scritto direttamente; `svc` fa salvare r0–r3, r12, lr, pc, xPSR sullo stack e il kernel scrive il risultato nel r0 salvato; `sys_exit` e `task_exit` non tornano

1.16.7 Programmi

programs/README.md — Nota di una riga: la cartella accoglie sorgenti e input dei programmi caricabili, passata a `tools/mkimage.py`.

- **Contiene:** solo testo Markdown
- **Dipende da:** `tools/mkimage.py` (scritto dagli studenti, non presente nella base)
- **Usato da:** `Makefile` (`PROGRAM_DIR`, target `image`); non è in `SRC_DIRS`, quindi `make` non la compila
- **Tocca:** niente

1.16.8 Mappa finale delle dipendenze

```
DIPENDENZE (file -> file; * = non presente nella base)
Makefile     -> mps2_m3.ld, tools/mkimage.py*, programs/*
mps2_m3.ld   -> startup.s (Reset_Handler, .isr_vector), idle.o, shell.o*
startup.s    -> mps2_m3.ld (simboli), main.c, svc_entry.s, pendsv.s, fault_entry.s, systick.c,
main.c       -> uart.h, systick.h, kprintf.h, mpu.h  (svc #5 scritto a mano, = SYS_START_TASKS
idle.c       -> src/tasks.h  |  irq.s -> nessuno  |  format.c -> format.h
systick.c    -> systick.h, cmsdk.h, kernel/task.h  |  uart.c -> uart.h, cmsdk.h, irq.h
task.c       -> kernel/task.h, cmsdk.h, mpu.h, kprintf.h, trace.h, src/tasks.h, lib/syscalls.h
syscall.c    -> kernel/syscall.h, lib/syscalls.h, uart.h, systick.h, kernel/task.h, trace.h
svc_entry.s  -> syscall.c  |  pendsv.s -> task.c  |  fault_entry.s -> faults.c
faults.c     -> cmsdk.h, kprintf.h, task.h  |  trace.c -> trace.h, task.h, kprintf.h
kprintf.c    -> kprintf.h, format.h, uart.h  |  mpu.c -> mpu.h, cmsdk.h
printf.c     -> printf.h, format.h, lib/syscalls.h  |  syscalls.s -> numeri di syscalls.h (a m

CATENE
BOOT:    Reset_Handler -> copia .data / azzera .bss -> VTOR, SHCSR, CCR, priorita' -> main ->
        uart_init, kprintf, systick_init, mpu_init -> svc #5 -> SVC_Handler -> svc_dispatch ->
        task_start_first -> idle (o shell)
TICK:    SysTick_Handler -> task_increment_tick -> task_request_switch (ICSR.PENDSVSET) ->
        PendSV_Handler -> task_switch_context -> mpu_set_task_region + trace_record_switch ->
SYSCALL: syscalls.s (svc #n) -> SVC_Handler -> svc_dispatch -> table[n] -> uart_write / task_*
        yield, delay, exit -> task_request_switch -> PendSV (come nella catena TICK)
FAULT:   *_Handler (fault_entry.s) -> fault_dispatch -> kprintf ; da programma caricato:
        task_fault_current -> complete_program -> PendSV ; altrimenti halt()
```